

L3: Building backtest framework and rule-based trading strategy

Course Code: COMP7415

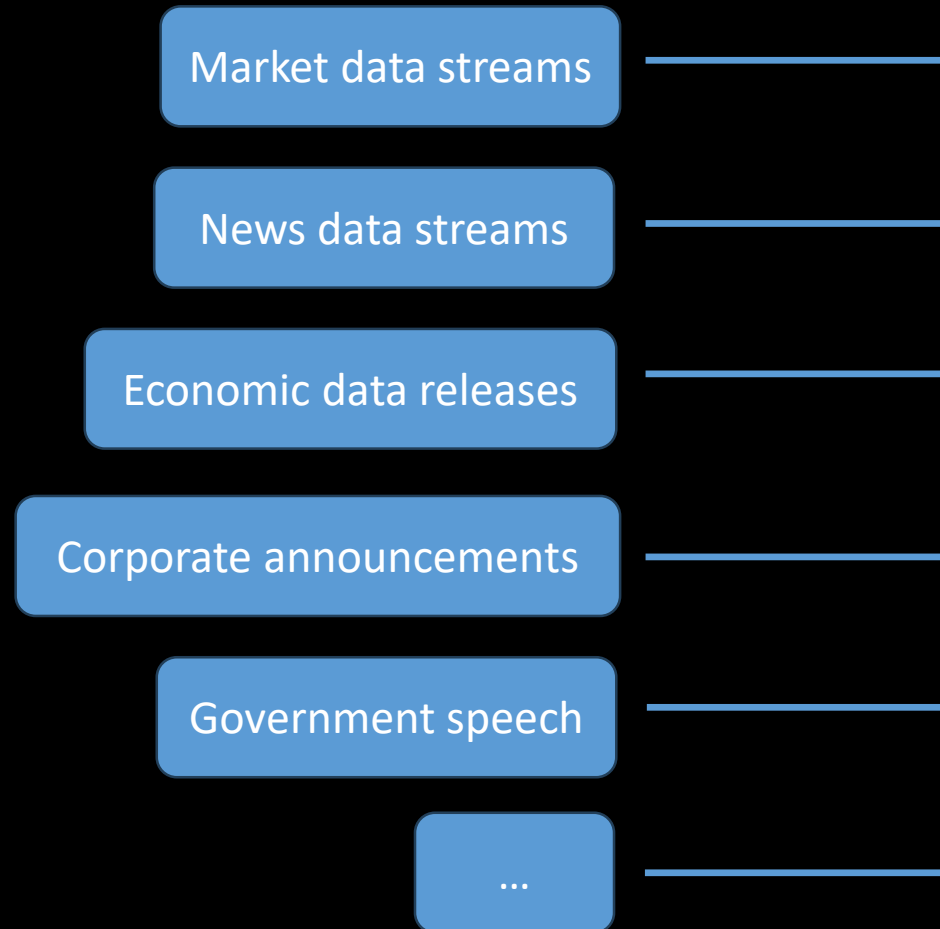
Agenda

- Introduction to Backtesting
- Common Candlestick Patterns
- Data Cleaning Techniques
- Implement an MA Cross Strategy on
 - Excel
 - Python
 - ALGOGENE
- Common Backtest Pitfalls
- Exercise: RSI Strategy

What is Backtesting?

- Definition: Simulating a trading strategy using historical data
- Purpose: Validate the effectiveness of a trading strategy before live trading
- Key Procedures:
 1. Data Collection
 2. Data Cleaning
 3. Strategy Implementation
 4. Performance Evaluation

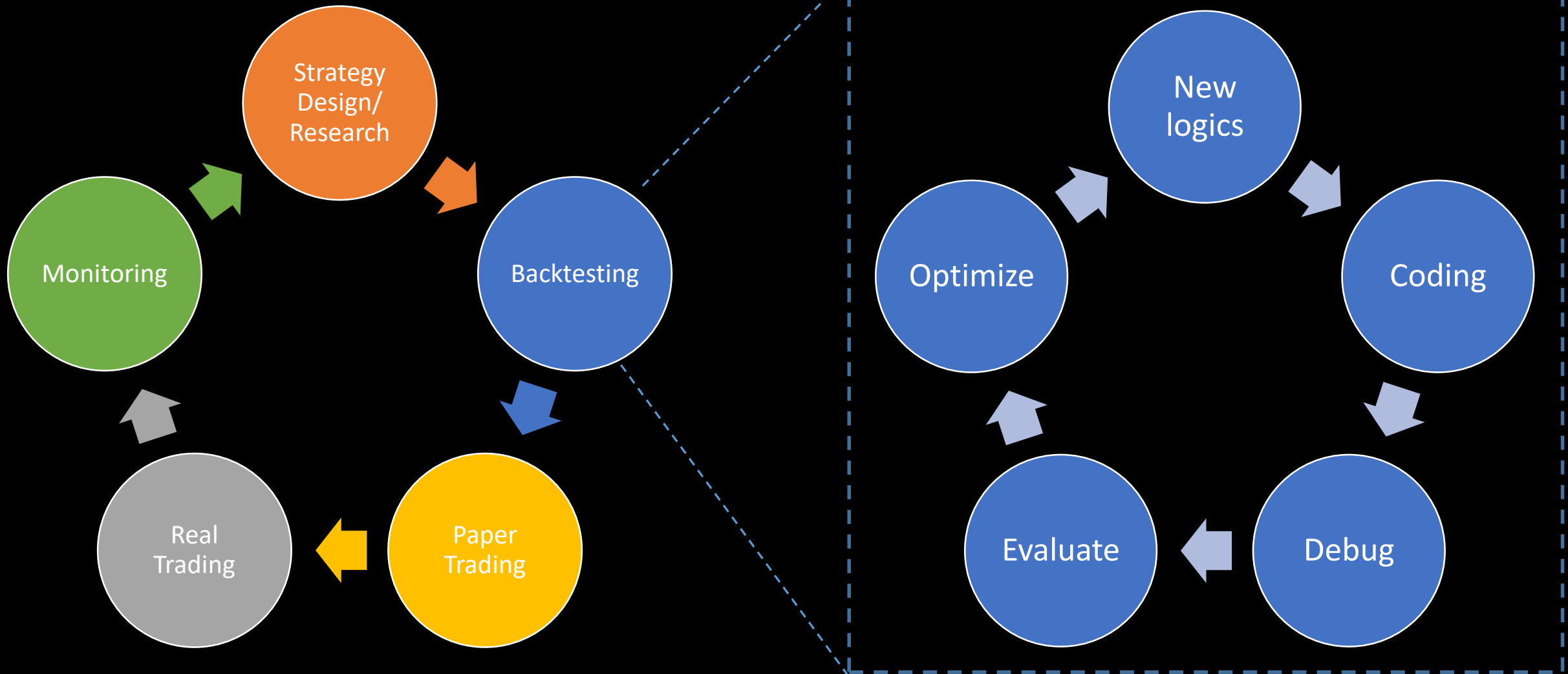
Backtesting is just like...



Why data cleaning?

- Remove data entry errors or system bugs from data provider
- Increase data quality and consistency
 - Easier for data management
 - Produces a more accurate statistical model
 - Better trade decision making
- Potential data issues:
 - Missing values
 - Duplicated records
 - Data contains incorrect logics

Algo Trading Lifecycle



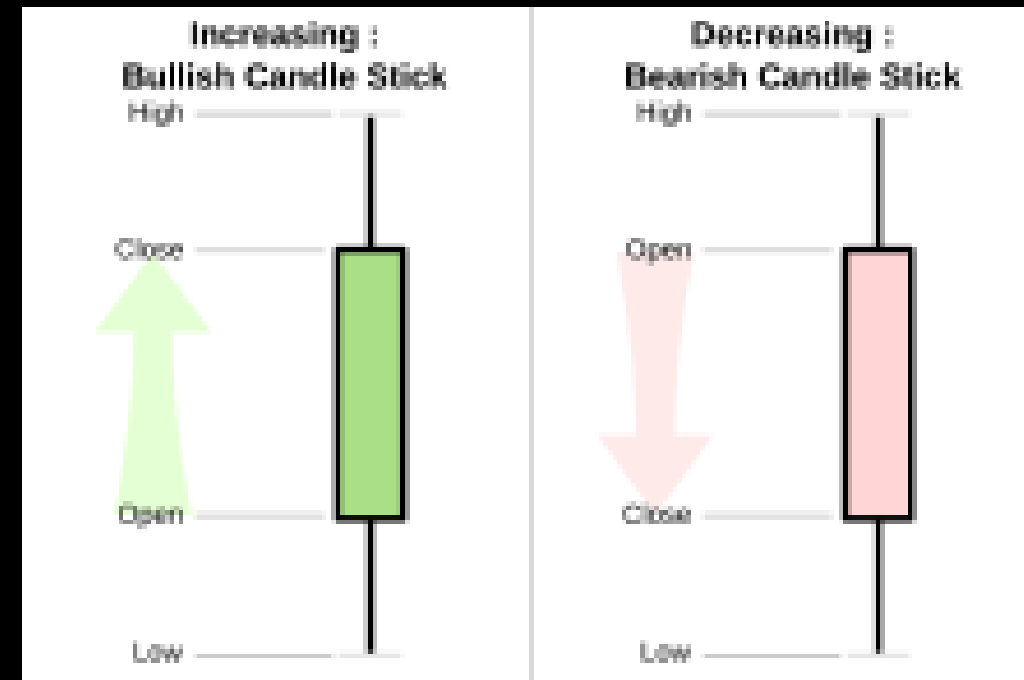
Candlestick

Introduction to Candlestick

- A candlestick chart (also called K-line) is a type of financial chart that displays the price movement of an asset over time.
- It summarizes OHLC data in a single chart
 - Open: The price at the start of the time period.
 - Close: The price at the end of the time period.
 - High: The highest price during the period.
 - Low: The lowest price during the period.
- OHLC data is aggregated in different time intervals (eg. 1-min, 1 hour, 1-day)
- In Technical Analysis, a candlestick pattern could provide indication on the future direction.

Understanding Candlestick Components

- Components:
 - **Body:** The area between the opening and closing prices.
 - **Wicks:** Lines extending above and below the body, representing the high and low prices.
- Visual display:
 - Bullish Candle: $\text{Close} > \text{Open}$ (often colored green or white)
 - Bearish Candle: $\text{Close} < \text{Open}$ (often colored red or black)



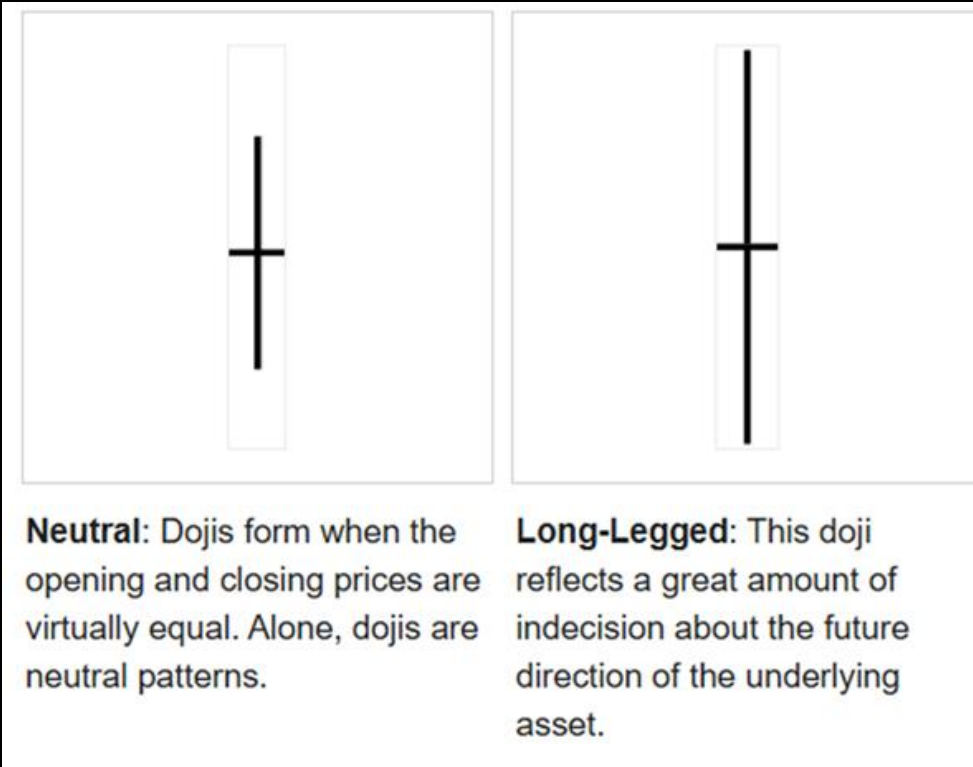
Common Candlestick Patterns

1. Doji
2. Hammer
3. Shooting Star
4. Double Bottom
5. Double Top
6. Head and Shoulders
7. Bull/Bear Flag
8. Bull/Bear Pennant
9. Ascending/Descending Triangle

Common Candlestick Patterns

1. Doji

- **Description:** A Doji candlestick forms when the open and close prices are nearly equal, creating a small body.
- **Market Insight:** The Doji indicates indecision in the market. Traders are uncertain about price direction, which can precede a reversal or continuation of the trend.
- **Example:** If a Doji appears after a bullish trend, it may signal a potential reversal, prompting traders to reevaluate their positions.

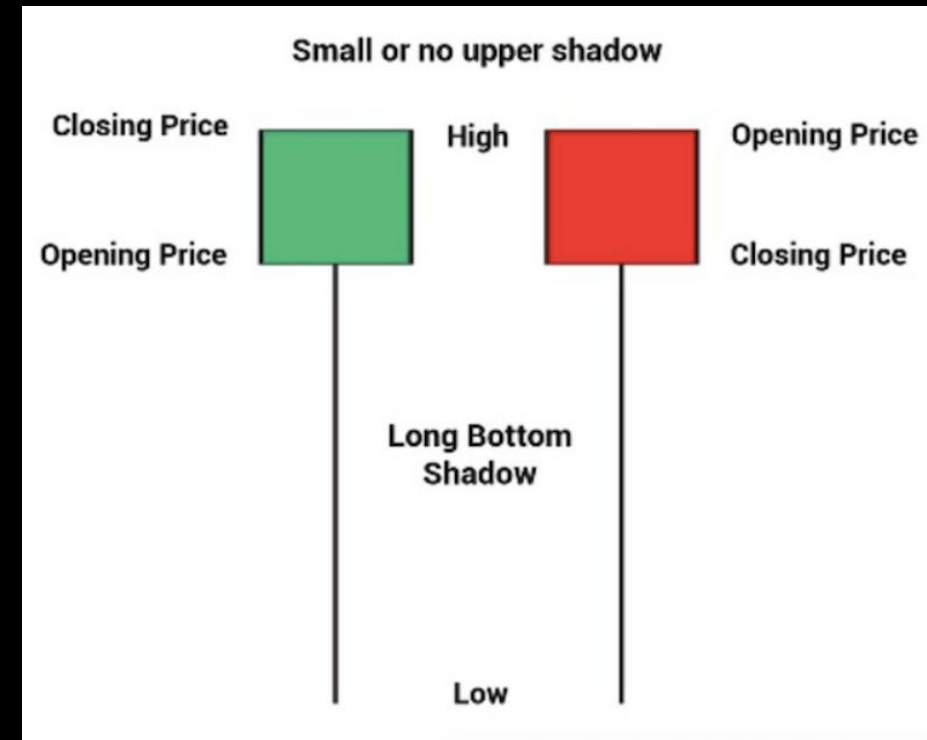




Common Candlestick Patterns

2. Hammer

- **Description:** The Hammer pattern consists of a long lower wick and a small body at the top, appearing after a downtrend.
- **Market Insight:** This pattern suggests a bullish reversal. The long lower wick indicates that sellers pushed prices down, but buyers stepped in, driving the price back up.
- **Example:** When a Hammer forms at a support level, it can indicate strong buying interest, signaling traders to consider long positions.

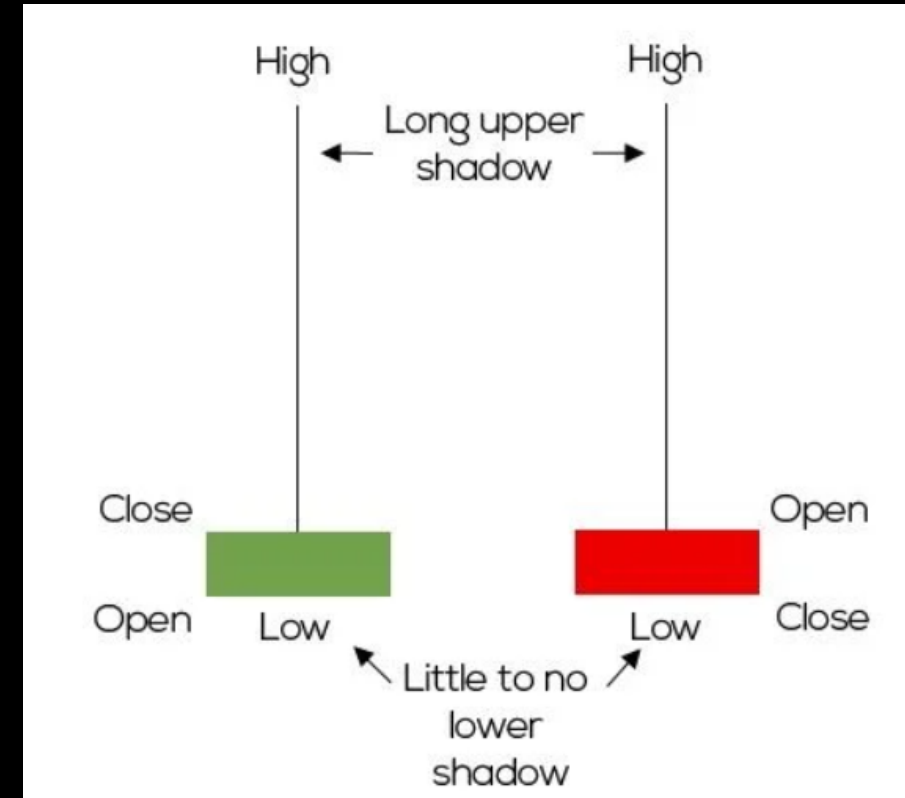




Common Candlestick Patterns

3. Shooting Star

- **Description:** The Shooting Star features a long upper wick with a small body at the bottom, forming after an uptrend.
- **Market Insight:** This pattern indicates a potential bearish reversal. The long upper wick shows that buyers pushed prices higher, but sellers took control, pushing the price back down.
- **Example:** If a Shooting Star appears at a resistance level, it may signal traders to consider short positions.





Common Candlestick Patterns

4. Double Bottom

- **Description:**

- A Double Bottom is a bullish reversal pattern that occurs after a downtrend.
- It consists of two troughs at approximately the same price level, with a peak in between.

- **Market Insight:**

- The first trough represents strong selling pressure, while the second trough indicates a potential reversal as buyers begin to emerge.
- A breakout above the peak confirms the pattern, signaling potential price appreciation.

- **Example:** Traders often look for confirmation with increased volume during the breakout to enhance reliability.



Common Candlestick Patterns

5. Double Top

- **Description:**

- A Double Top is a bearish reversal pattern that occurs after an uptrend.
- It consists of two peaks at roughly the same price level, with a trough in between.

- **Market Insight:**

- The first peak indicates strong buying pressure, but the second peak shows weakening momentum as sellers begin to take control.
- A breakout below the trough confirms the pattern, signaling a potential price decline.

- **Example:** As with the Double Bottom, increased volume during the breakout can enhance the pattern's validity.



There are so many other patterns...

Head and Shoulders



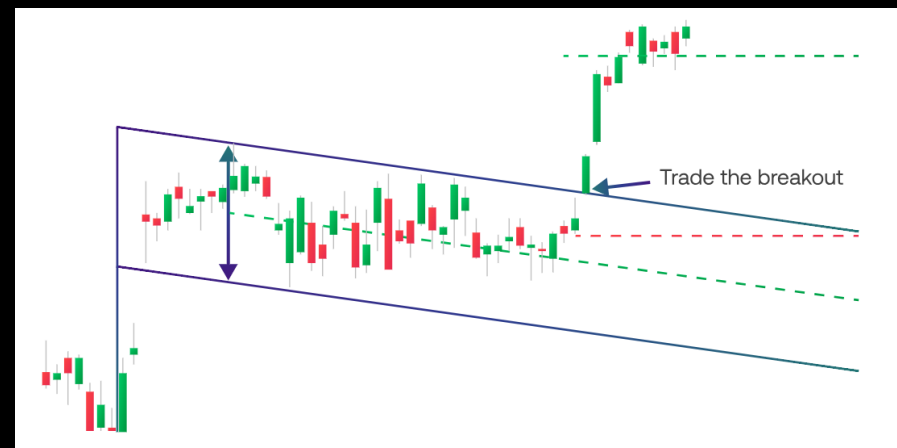
Ascending Triangle



Pennant



Flag



Backtest with Excel

Why use Excel for backtest?

- **Accessibility:** Widely available and user-friendly
- **Visualization:** Easy to create charts
- **Simplicity:** No programming knowledge required for basic tasks

Download Data

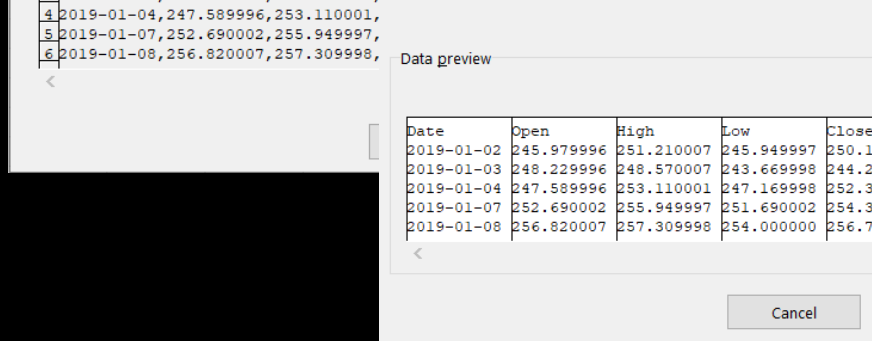
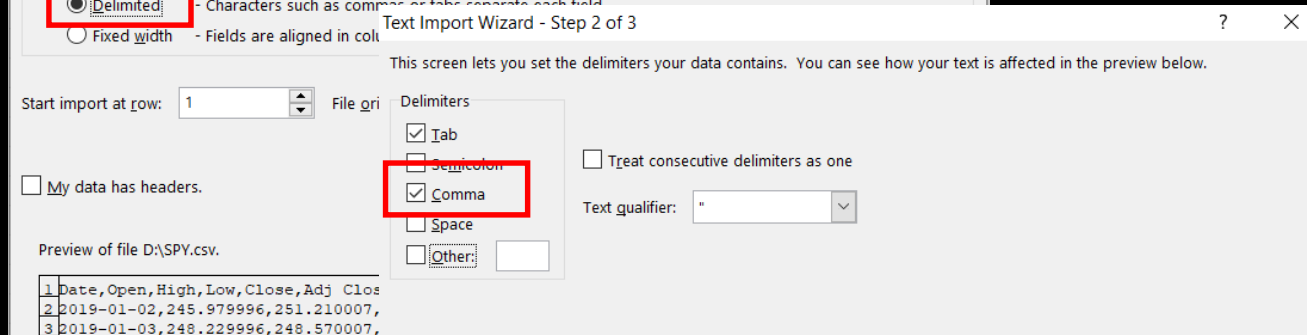
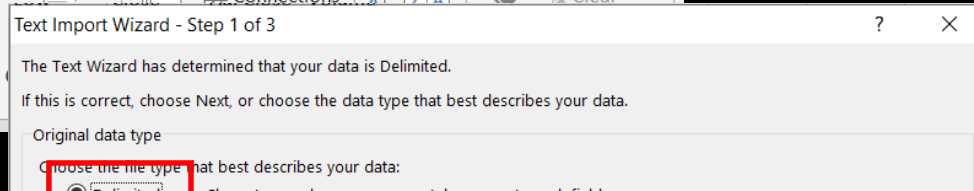
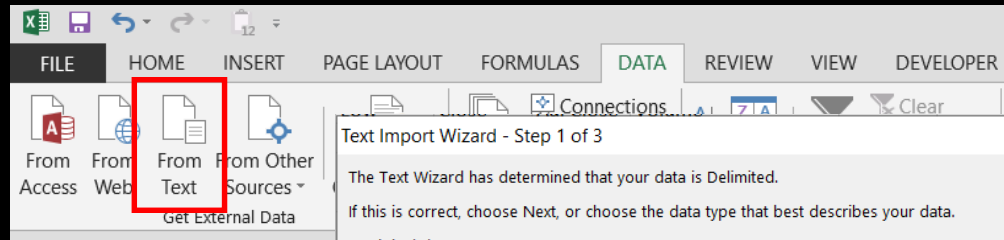
<https://finance.yahoo.com/quote/0005.HK/history/?period1=1546300800&period2=1703980800>

- Instrument: 0005.HK
- Period: 2010.01.01 – 2023.12.31

The screenshot shows the Yahoo Finance page for HSBC HOLDINGS (0005.HK). The page includes a search bar, navigation tabs (My Portfolio, News, Markets, Research, Personal Finance, Videos, Streaming Now), and a sidebar with links to Summary, News, Chart, Conversations, Statistics, Historical Data (highlighted), Profile, Financials, Analysis, Options, Holders, and Sustainability. The main content area displays the stock price at 79.950, up 0.500 (+0.63%) as of January 28 at 12:08:04 PM GMT+8. It also features a 'Start Trading >>' button and a 'Plus500 CFD Service' warning. Below this, there are dropdown menus for the date range (Jan 01, 2019 - Dec 31, 2023), historical prices, and daily view. The table below shows the historical price data for the period from December 14, 2023, to December 29, 2023.

Date	Open	High	Low	Close ①	Adj Close ①	Volume
Dec 29, 2023	63.000	63.300	62.750	63.000	58.339	11,249,734
Dec 28, 2023	63.000	63.300	62.850	63.150	58.478	23,679,787
Dec 27, 2023	62.500	62.800	62.050	62.550	57.923	17,552,636
Dec 22, 2023	61.400	62.000	61.250	61.800	57.228	18,340,964
Dec 21, 2023	60.500	61.300	60.500	61.250	56.719	7,873,148
Dec 20, 2023	60.600	61.350	60.600	61.200	56.673	14,560,834
Dec 19, 2023	60.150	60.500	59.900	60.300	55.839	6,039,241
Dec 18, 2023	60.500	60.500	60.000	60.350	55.885	15,270,566
Dec 15, 2023	61.000	61.600	60.900	61.350	56.811	18,382,310
Dec 14, 2023	61.000	61.350	60.700	61.000	56.487	15,025,329

Import Data to Excel



	A	B	C	D	E	F	G	H	I
1	Date	Open	High	Low	Close	Adj Close	Volume		
2	04/01/2010	89.4	89.9	88.8	89.25	44.81234	10381759		
3	05/01/2010	90.2	90.65	90.1	90.45	45.41485	16914189		
4	06/01/2010	91.3	92	91.25	91.6	45.99229	20033996		
5	07/01/2010	91.7	91.75	90.85	91.25	45.81654	11163952		
6	08/01/2010	91.5	92.15	91.35	91.7	46.04247	15683744		
7	11/01/2010	92	92.8	92	92.4	46.39394	15729515		
8	12/01/2010	91.8	92.4	91.05	91.65	46.01737	14005577		
9	13/01/2010	90.5	90.8	89.95	90.2	45.28933	17142233		
10	14/01/2010	90.65	91.2	90.65	90.85	45.61569	11379186		
11	15/01/2010	91	90.3	90.75	91.05	45.71613	13190937		
12	18/01/2010	89.05	89.6	88.6	89.25	44.81234	16529101		

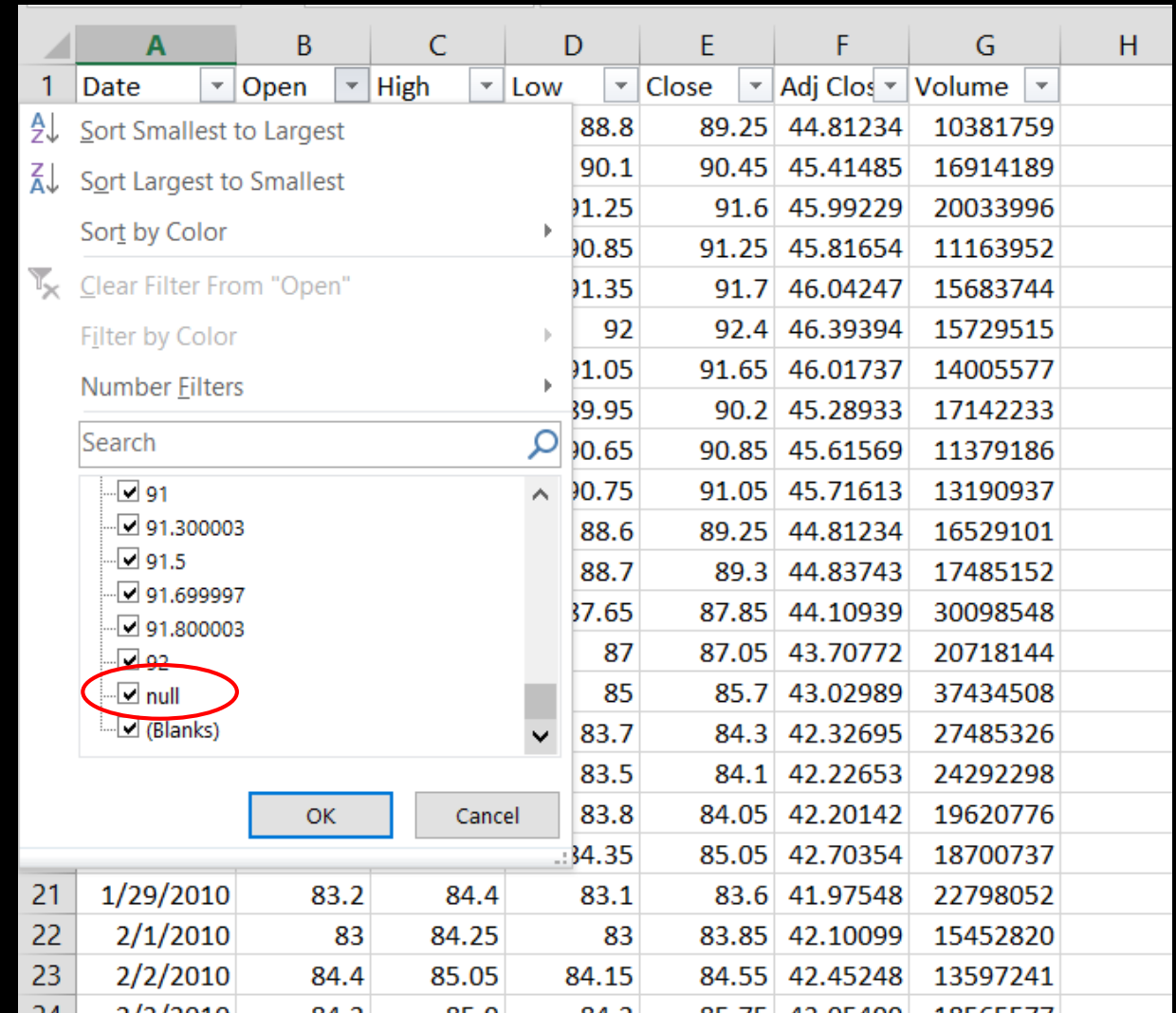
Data Cleaning (1): missing data

Objective:

- Search for rows with missing value

How to solve?

- Check from other data sources (if available)
- Fill in the missing value based on adjacent observations (eg. average, smoothing, etc)
- Delete the rows



The screenshot shows an Excel spreadsheet with columns A through H. The data table starts at row 1. A filter menu is open over column D, showing a list of values. The 'null' value is circled in red.

	A	B	C	D	E	F	G	H
1	Date	Open	High	Low	Close	Adj Close	Volume	
				88.8	89.25	44.81234	10381759	
				90.1	90.45	45.41485	16914189	
				91.25	91.6	45.99229	20033996	
				90.85	91.25	45.81654	11163952	
				91.35	91.7	46.04247	15683744	
				92	92.4	46.39394	15729515	
				91.05	91.65	46.01737	14005577	
				89.95	90.2	45.28933	17142233	
				90.65	90.85	45.61569	11379186	
				90.75	91.05	45.71613	13190937	
				88.6	89.25	44.81234	16529101	
				88.7	89.3	44.83743	17485152	
				87.65	87.85	44.10939	30098548	
				87	87.05	43.70772	20718144	
				85	85.7	43.02989	37434508	
				83.7	84.3	42.32695	27485326	
				83.5	84.1	42.22653	24292298	
				83.8	84.05	42.20142	19620776	
				84.35	85.05	42.70354	18700737	
21	1/29/2010	83.2	84.4	83.1	83.6	41.97548	22798052	
22	2/1/2010	83	84.25	83	83.85	42.10099	15452820	
23	2/2/2010	84.4	85.05	84.15	84.55	42.45248	13597241	
24	2/3/2010	84.2	85.0	84.2	85.75	42.95400	18565577	

Data Cleaning (2): duplicated records

Objective:

- Define our key (eg. Date)
- Count the occurrence of key

How to solve?

- if the duplicated records are completely identical,
 - Keep only 1 of them
- If NOT identical,
 - Check from other data sources to determine which one is correct (if available)
 - Delete all duplicated records
 - Take average of them

	A	B	C	D	E	F	G	H
	Date	Open	High	Low	Close	Adj Close	Volume	count
2	1/4/2010	89.4	89.9	88.8	89.25	44.81234	10381759	1
3	1/5/2010	90.2	90.65	90.1	90.45	45.41485	16914189	1
4	1/6/2010	91.3	92	91.25	91.6	45.99229	20033996	1
5	1/7/2010	91.7	91.75	90.85	91.25	45.81654	11163952	1
6	1/8/2010	91.5	92.15	91.35	91.7	46.04247	15683744	1
7	1/11/2010	92	92.8	92	92.4	46.39394	15729515	1
8	1/12/2010	91.8	92.4	91.05	91.65	46.01737	14005577	1
9	1/13/2010	90.5	90.8	89.95	90.2	45.28933	17142233	1
10	1/14/2010	90.65	91.2	90.65	90.85	45.61569	11379186	1
11	1/15/2010	91	90.3	90.75	91.05	45.71613	13190937	1
12	1/18/2010	89.05	89.6	88.6	89.25	44.81234	16529101	1
13	1/19/2010	89.05	88.4	88.7	89.3	44.83743	17485152	1

Data Cleaning (3): incorrect logics

Objective:

- A valid row should satisfy the following conditions
 - Close $\leq$ High and Close $\geq$ Low
 - Open $\leq$ High and Open $\geq$ Low

How to solve?

- Check from other data sources (if available)
- Delete the rows
- Cap and floor the value with High and Low respectively

=AND(E2<=C2, E2>=D2)									
	A	B	C	D	E	F	G	H	I
1	Date	Open	High	Low	Close	Adj Close	Volume	C<=H and C>	O<=H and O>
2	1/4/2010	89.4	89.9	88.8	89.25	44.81234	10381759	TRUE	TRUE
3	1/5/2010	90.2	90.65	90.1	90.45	45.41485	16914189	TRUE	TRUE
4	1/6/2010	91.3	92	91.25	91.6	45.99229	20033996	TRUE	TRUE
5	1/7/2010	91.7	91.75	90.85	91.25	45.81654	11163952	TRUE	TRUE
6	1/8/2010	91.5	92.15	91.35	91.7	46.04247	15683744	TRUE	TRUE
7	1/11/2010	92	92.8	92	92.4	46.39394	15729515	TRUE	TRUE
8	1/12/2010	91.8	92.4	91.05	91.65	46.01737	14005577	TRUE	TRUE

[illegible]

Moving Average Strategy

Moving Average

- **Definition:** A moving average (MA) is a statistical calculation used to analyze data points by creating a series of averages of different subsets of the full data set.
- **Purpose:** To smooth out short-term fluctuations and highlight longer-term trends in data.
- **Types of Moving Averages:**
 - Simple Moving Average (SMA)
 - Exponential Moving Average (EMA)

Simple Moving Average (SMA)

- The average price over a specific number of periods n .
- Formula:

$$MA(n) = \frac{\sum_{t=1}^n P_t}{n}$$

- Example:
 - Given Prices: [10, 12, 14, 16, 18]
 - 5-Day SMA Calculation:

$$SMA = \frac{10 + 12 + 14 + 16 + 18}{5} = 14$$

Exponential Moving Average (EMA)

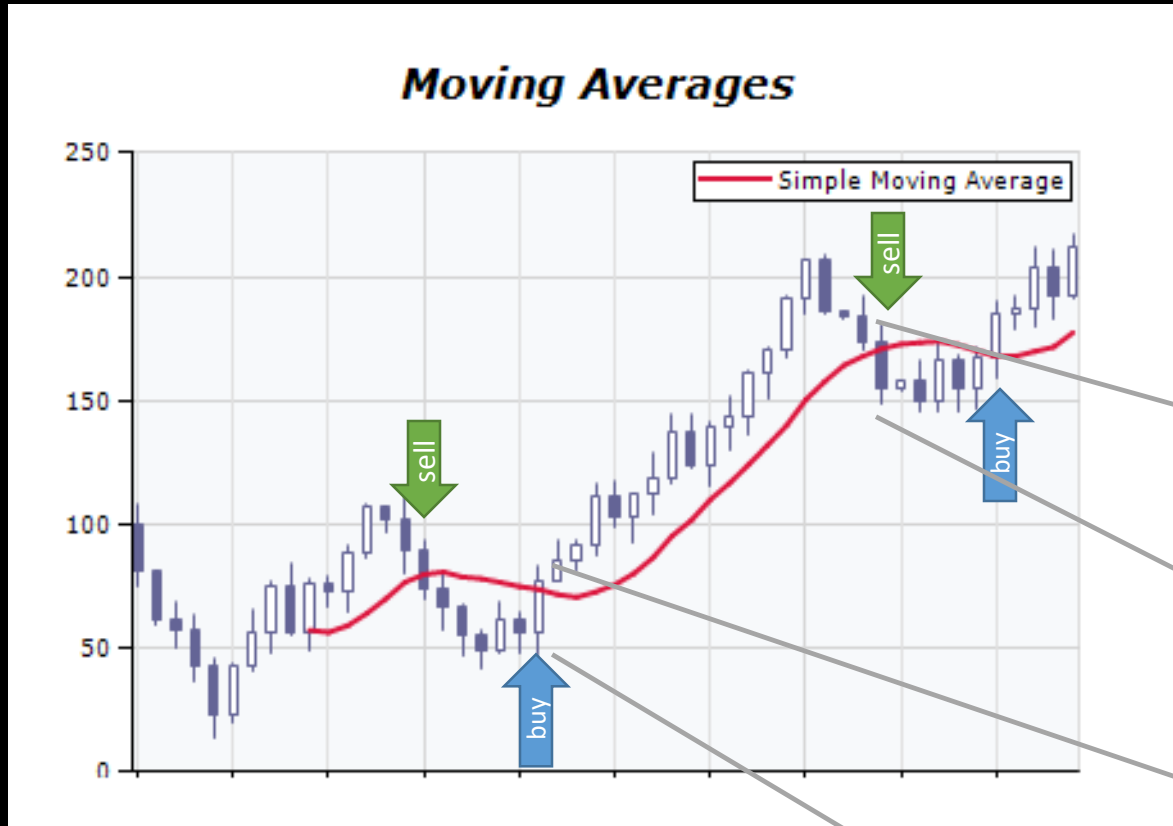
- Gives more weight to recent prices, making it more responsive to new information
- Formula:

$$EMA_t = P_t \times k + (1 - k) \times EMA_{t-1}$$

Commonly chosen

$$\begin{cases} k = \frac{2}{n+1} \\ EMA_0 = P_0 \end{cases}$$

Simple MA Strategy

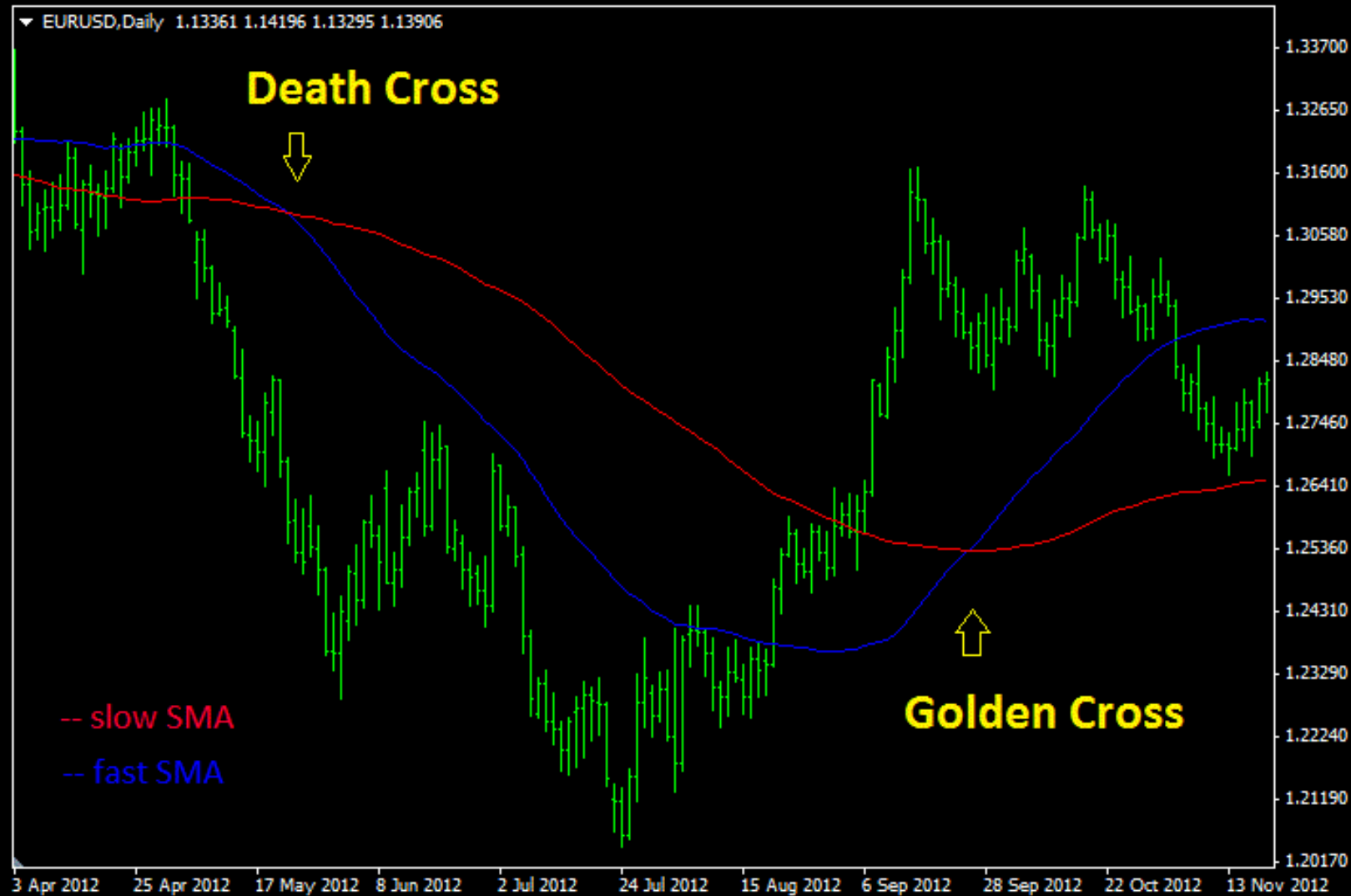


Trading Logic:

1. if $\text{Open} < \text{MA}$ and $\text{Close} > \text{MA}$, then BUY
2. If $\text{Open} > \text{MA}$ and $\text{Close} < \text{MA}$, then SELL



MA Cross Strategy



MA Cross Strategy

- Define
 - fast MA as 7-day moving average
 - slow MA as 14-day moving average
- Based on a sliding window approach to collect the previous 14 closing price
 - Calculate fast MA and slow MA values
 - Open order conditions:
 - **Golden Cross:** BUY if $\text{fast MA}(t-1) < \text{slow MA}(t-1)$ AND $\text{fast MA}(t) > \text{slow MA}(t)$, OR
 - **Death Cross:** SELL if $\text{fast MA}(t-1) > \text{slow MA}(t-1)$ AND $\text{fast MA}(t) < \text{slow MA}(t)$
 - Close order conditions:
 - If previous BUY and now Death Cross appears, OR
 - If previous SELL and now Golden Cross appears
- Repeat the same process until the backtest period end

Backtest Assumptions

1. No transaction cost
2. Buy/sell 1 share each time
3. Order entry/exit at market close
4. Short selling is NOT allowed

MA Cross Strategy in Excel

1. Calculate MA(7) and MA(14)

$$SMA_k = \frac{p_{n-k+1} + p_{n-k+2} + \dots + p_n}{k}$$

SUM				✕		✓		fx		=AVERAGE(F2:F8)			
	A	B	C	D	E	F	G	H	I	J			
1	Date	Open	High	Low	Close	Adj Close	Volume	MA(7)	MA(14)				
2	1/4/2010	89.4	89.9	88.8	89.25	44.81234	10381759						
3	1/5/2010	90.2	90.65	90.1	90.45	45.41485	16914189						
4	1/6/2010	91.3	92	91.25	91.6	45.99229	20033996						
5	1/7/2010	91.7	91.75	90.85	91.25	45.81654	11163952						
6	1/8/2010	91.5	92.15	91.35	91.7	46.04247	15683744						
7	1/11/2010	92	92.8	92	92.4	46.39394	15729515						
8	1/12/2010	91.8	92.4	91.05	91.65	46.01737	14005577	=AVERAGE(F2:F8)					
9	1/13/2010	90.5	90.8	89.95	90.2	45.28933	17142233	45.8524					
10	1/14/2010	90.65	91.2	90.65	90.85	45.61569	11379186	45.88109					
11	1/18/2010	89.05	89.6	88.6	89.25	44.81234	16529101	45.71253					
12	1/20/2010	88	88.45	87.65	87.85	44.10939	30098548	45.46865					
13	1/21/2010	87.6	88.2	87	87.05	43.70772	20718144	45.13511					
14	1/22/2010	85.5	86.05	85	85.7	43.02989	37434508	44.65453					
15	1/25/2010	84	84.35	83.7	84.3	42.32695	27485326	44.12733	44.95579				
16	1/26/2010	84.6	85.15	83.5	84.1	42.22653	24292298	43.68979	44.77109				
17	1/27/2010	84	85.6	83.8	84.05	42.20142	19620776	43.20203	44.54156				
18	1/28/2010	84.35	85.65	84.35	85.05	42.70354	18700737	42.90078	44.30665				
19	1/29/2010	83.2	84.4	83.1	83.6	41.97548	22798052	42.59593	44.03229				
20	2/1/2010	83	84.25	83	83.85	42.10099	15452820	42.3664	43.75076				
21	2/2/2010	84.4	85.95	84.15	84.55	42.45248	12597241	42.28201	43.46922				

Simple Moving Average



MA Cross Strategy in Excel

2. Identify MA cross signal

Buy signal

- $MA(7,t) > MA(14,t)$ and $MA(7,t-1) < MA(14,t-1)$

Sell signal

- $MA(7,t) < MA(14,t)$ and $MA(7,t-1) > MA(14,t-1)$

J16	:				=IF(AND(H16>I16,H15<I15), 1, IF(AND(H16<I16,H15>I15), -1, 0))							
	A	B	C	D	E	F	G	H	I	J		
1	Date	Open	High	Low	Close	Adj Clos	Volume	MA(7)	MA(14)	signal		
2	1/4/2010	89.4	89.9	88.8	89.25	44.81234	10381759					
3	1/5/2010	90.2	90.65	90.1	90.45	45.41485	16914189					
4	1/6/2010	91.3	92	91.25	91.6	45.99229	20033996					
5	1/7/2010	91.7	91.75	90.85	91.25	45.81654	11163952					
6	1/8/2010	91.5	92.15	91.35	91.7	46.04247	15683744					
7	1/11/2010	92	92.8	92	92.4	46.39394	15729515					
8	1/12/2010	91.8	92.4	91.05	91.65	46.01737	14005577	45.78426				
9	1/13/2010	90.5	90.8	89.95	90.2	45.28933	17142233	45.8524				
10	1/14/2010	90.65	91.2	90.65	90.85	45.61569	11379186	45.88109				
11	1/18/2010	89.05	89.6	88.6	89.25	44.81234	16529101	45.71253				
12	1/20/2010	88	88.45	87.65	87.85	44.10939	30098548	45.46865				
13	1/21/2010	87.6	88.2	87	87.05	43.70772	20718144	45.13511				
14	1/22/2010	85.5	86.05	85	85.7	43.02989	37434508	44.65453				
15	1/25/2010	84	84.35	83.7	84.3	42.32695	27485326	44.12733	44.95579			
16	1/26/2010	84.6	85.15	83.5	84.1	42.22653	24292298	43.68979	44.77109	0		
17	1/27/2010	84	85.6	83.8	84.05	42.20142	19620776	43.20203	44.54156	0		
18	1/28/2010	84.35	85.65	84.35	85.05	42.70354	18700737	42.90078	44.30665	0		
19	1/29/2010	83.2	84.4	83.1	83.6	41.97548	22798052	42.59593	44.03229	0		
20	2/1/2010	83	84.25	83	83.85	42.10099	15452820	42.3664	43.75076	0		

MA Cross Strategy in Excel

3. Calculate day-end holding

=K2+IF(AND(K2=0,J3=-1), 0, J3)												
	A	B	C	D	E	F	G	H	I	J	K	
1	Date	Open	High	Low	Close	Adj Close	Volume	MA(7)	MA(14)	signal	Position	
2	1/4/2010	89.4	89.9	88.8	89.25	44.81234	10381759				0	
3	1/5/2010	90.2	90.65	90.1	90.45	45.41485	16914189				0	
4	1/6/2010	91.3	92	91.25	91.6	45.99229	20033996				0	
5	1/7/2010	91.7	91.75	90.85	91.25	45.81654	11163952				0	
6	1/8/2010	91.5	92.15	91.35	91.7	46.04247	15683744				0	
7	1/11/2010	92	92.8	92	92.4	46.39394	15729515				0	
8	1/12/2010	91.8	92.4	91.05	91.65	46.01737	14005577	45.78426			0	
9	1/13/2010	90.5	90.8	89.95	90.2	45.28933	17142233	45.8524			0	
10	1/14/2010	90.65	91.2	90.65	90.85	45.61569	11379186	45.88109			0	
11	1/18/2010	89.05	89.6	88.6	89.25	44.81234	16529101	45.71253			0	
12	1/20/2010	88	88.45	87.65	87.85	44.10939	30098548	45.46865			0	
13	1/21/2010	87.6	88.2	87	87.05	43.70772	20718144	45.13511			0	
14	1/22/2010	85.5	86.05	85	85.7	43.02989	37434508	44.65453			0	
15	1/25/2010	84	84.35	83.7	84.3	42.32695	27485326	44.12733	44.95579		0	
16	1/26/2010	84.6	85.15	83.5	84.1	42.22653	24292298	43.68979	44.77109	0	0	
17	1/27/2010	84	85.6	83.8	84.05	42.20142	19620776	43.20203	44.54156	0	0	
18	1/28/2010	84.35	85.65	84.35	85.05	42.70354	18700737	42.90078	44.30665	0	0	
19	1/29/2010	83.2	84.4	83.1	83.6	41.97548	22798052	42.59593	44.03229	0	0	

MA Cross Strategy in Excel

4. Calculate daily PnL

L3															
	A	B	C	D	E	F	G	H	I	J	K	L	M		
1	Date	Open	High	Low	Close	Adj Close	Volume	MA(7)	MA(14)	signal	Position	Daily PnL	Cum PnL		
2	1/4/2010	89.4	89.9	88.8	89.25	44.81234	10381759				0				
3	1/5/2010	90.2	90.65	90.1	90.45	45.41485	16914189				0	0			
4	1/6/2010	91.3	92	91.25	91.6	45.99229	20033996				0	0			
5	1/7/2010	91.7	91.75	90.85	91.25	45.81654	11163952				0	0			
6	1/8/2010	91.5	92.15	91.35	91.7	46.04247	15683744				0	0			
7	1/11/2010	92	92.8	92	92.4	46.39394	15729515				0	0			
8	1/12/2010	91.8	92.4	91.05	91.65	46.01737	14005577	45.78426			0	0			

5. Calculate cumulative PnL

M3															
	A	B	C	D	E	F	G	H	I	J	K	L	M		
1	Date	Open	High	Low	Close	Adj Close	Volume	MA(7)	MA(14)	signal	Position	Daily PnL	Cum PnL		
2	1/4/2010	89.4	89.9	88.8	89.25	44.81234	10381759				0		0		
3	1/5/2010	90.2	90.65	90.1	90.45	45.41485	16914189				0	0	0		
4	1/6/2010	91.3	92	91.25	91.6	45.99229	20033996				0	0	0		
5	1/7/2010	91.7	91.75	90.85	91.25	45.81654	11163952				0	0	0		
6	1/8/2010	91.5	92.15	91.35	91.7	46.04247	15683744				0	0	0		
7	1/11/2010	92	92.8	92	92.4	46.39394	15729515				0	0	0		
8	1/12/2010	91.8	92.4	91.05	91.65	46.01737	14005577	45.78426			0	0	0		
9	1/13/2010	90.5	90.8	89.95	90.2	45.28933	17142233	45.8524			0	0	0		

Got a profitable strategy?



Does our assumptions make sense?

1. No transaction cost
 - in fact, there is transaction cost in reality (eg. commission, swap fee, funding cost, tax, etc)
2. Buy/sell 1 share each time
3. Order entry/exit at market close
 - can we really trade at the last moment before market close? Why not execute at market open?
4. Short selling is NOT allowed
 - short-selling stocks are not easily executable in the market due to
 - Regulation requirements
 - Some brokers require extra cash as collateral



HSBC HOLDINGS PLC · 1 · HKEX ● D ! O79.95 H79.95 L79.85 C79.85 -0.05 (-0.06%)

80.15 SELL 0.05 80.20 BUY

Vol 108K



TradingView

Feb 10:00 10:30 11:00 11:30 13:00 13:30 14:00 14:30 15:00
1D 5D 1M 3M 6M YTD 1Y 5Y All ↻

Mon 03 Feb '25 15:59 10:20:00 UTC+8 RTH ADJ

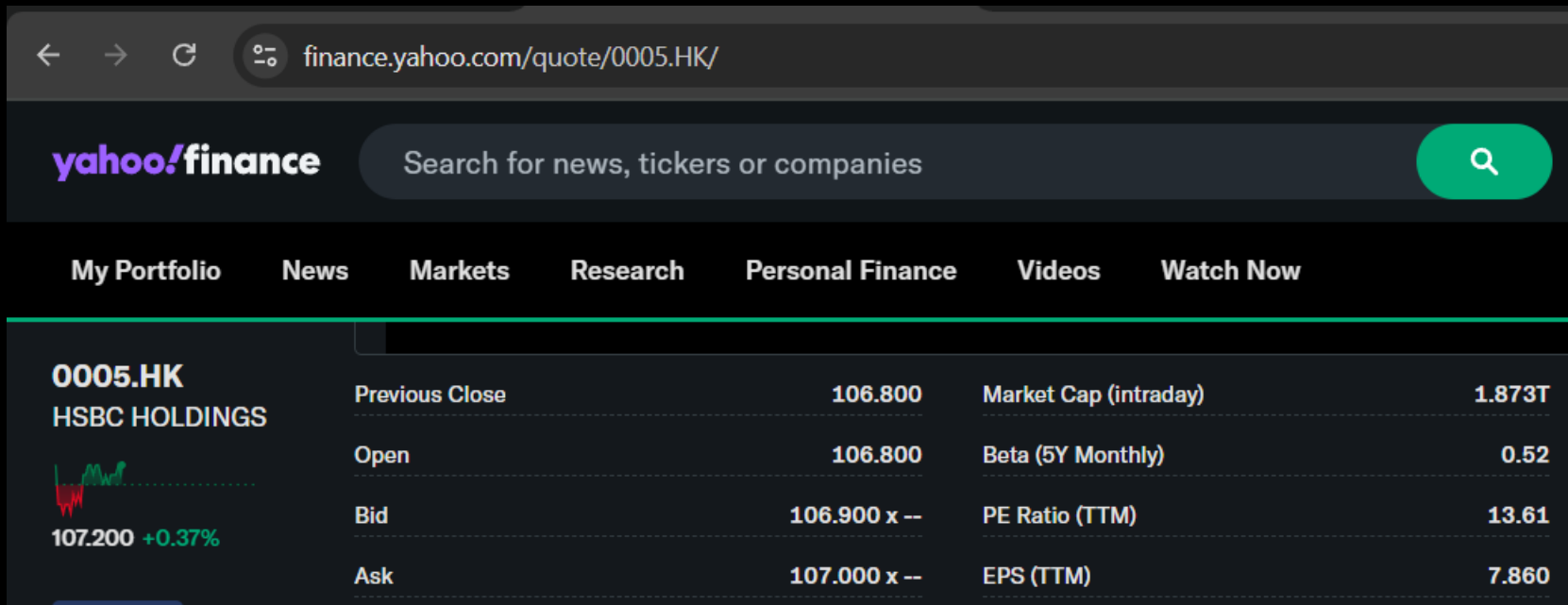
MA Cross Strategy in Excel

- Adjust for realistic assumptions
 - Bid ask spread for 0005.HK is usually around 0.1
 - HK stock stamp duty = 0.1%
 - Commission around 0.02%



Market price is around \$100 so stamp duty and commission is equivalent to $100 \times (0.1\% + 0.02\%) = 0.12$ point value. Adding the 0.1 spread cost will be 0.22

Let's assume 0.25 price point for conservative measure



MA Cross Strategy in Excel

=K2*(F3-F2) -IF(K2<>K3,0.25,0)																
	A	B	C	D	E	F	G	H	I	J	K	L	M			
1	Date	Open	High	Low	Close	Adj Clos	Volume	MA(7)	MA(14)	signal	Position	Daily PnL	Cum PnL			
2	4/1/2010	89.4	89.9	88.8	89.25	44.81234	10381759				0		0			
3	5/1/2010	90.2	90.65	90.1	90.45	45.41485	16914189				0	0	0			
4	6/1/2010	91.3	92	91.25	91.6	45.99229	20033996				0	0	0			
5	7/1/2010	91.7	91.75	90.85	91.25	45.81654	11163952				0	0	0			
6	8/1/2010	91.5	92.15	91.35	91.7	46.04247	15683744				0	0	0			
7	11/1/2010	92	92.8	92	92.4	46.39394	15729515				0	0	0			
8	12/1/2010	91.8	92.4	91.05	91.65	46.01737	14005577	45.78426			0	0	0			
9	13/1/2010	90.5	90.8	89.95	90.2	45.28933	17142233	45.8524			0	0	0			
10	14/1/2010	90.65	91.2	90.65	90.85	45.61569	11379186	45.88109			0	0	0			
11	18/1/2010	89.05	89.6	88.6	89.25	44.81234	16529101	45.71253			0	0	0			
12	20/1/2010	88	88.45	87.65	87.85	44.10939	30098548	45.46865			0	0	0			
13	21/1/2010	87.6	88.2	87	87.05	43.70772	20718144	45.13511			0	0	0			
14	22/1/2010	85.5	86.05	85	85.7	43.02989	37434508	44.65453			0	0	0			
15	25/1/2010	84	84.35	83.7	84.3	42.32695	27485326	44.12733	44.95579		0	0	0			
16	26/1/2010	84.6	85.15	83.5	84.1	42.22653	24292298	43.68979	44.77109	0	0	0	0			
17	27/1/2010	84	85.6	83.8	84.05	42.20142	19620776	43.20203	44.54156	0	0	0	0			
18	28/1/2010	84.35	85.65	84.35	85.05	42.70354	18700737	42.90078	44.30665	0	0	0	0			
19	29/1/2010	83.2	84.4	83.1	83.6	41.97548	22798052	42.59593	44.03229	0	0	0	0			

Result not good after factoring in costs

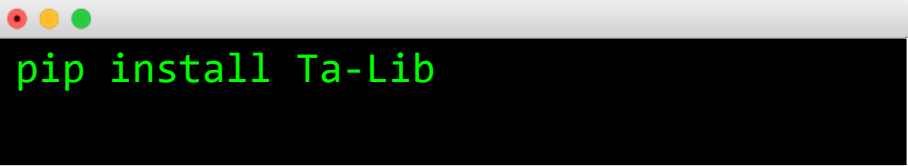


Backtest with Python

Technical Indicator

TA-LIB

- Official Document:
<https://pypi.org/project/TA-Lib/>
- An open source library with 120+ technical indicators, such as MA, RSI, Bollinger Bands, etc



```
pip install Ta-Lib
```

MA - Moving average

```
real = MA(close, timeperiod=30, matype=0)
```

RSI - Relative Strength Index

NOTE: The `RSI` function has an unstable period.

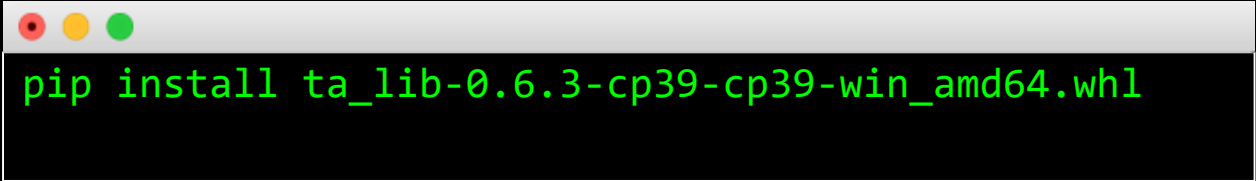
```
real = RSI(close, timeperiod=14)
```

BBANDS - Bollinger Bands

```
upperband, middleband, lowerband = BBANDS(close, timeperiod=5, nbdevup=2, nbdevdn=2, matype=
```

Alternative Installation Method

- Download the wheel file that match with your python version <https://github.com/cgohlke/talib-build/releases> (eg. ta_lib-0.6.3-cp39-cp39-win_amd64.whl)
- Then install from your downloaded directory

A terminal window with a light gray title bar containing three colored window control buttons (red, yellow, green). The terminal area has a black background with green text.

```
pip install ta_lib-0.6.3-cp39-cp39-win_amd64.whl
```

Example

- Given a list of price series [10, 11, 12, 15, 14, 13, 16, 17, 19, 17] and MA period = 5

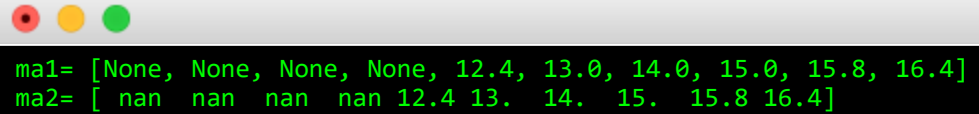
```
from talib import MA
import numpy as np

close = [10.0, 11.0, 12.0, 15.0, 14.0, 13.0, 16.0, 17.0, 19.0, 17.0]

def my_MA(arr, period):
    out = []
    for i in range(0, len(arr)):
        tmp = arr[max(0, i+1-period):i+1]
        if len(tmp) < period:
            out.append(None)
        else:
            value = sum(tmp)/period
            out.append(value)
    return out

ma1 = my_MA(close, 5)
ma2 = MA(np.array(close), timeperiod=5)

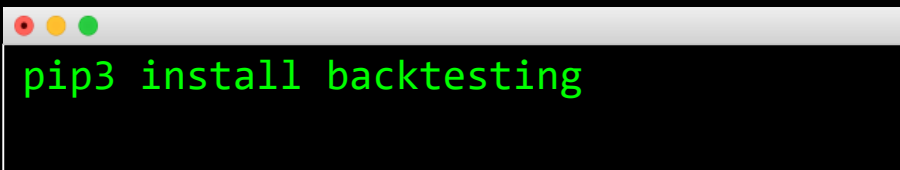
print("ma1=", ma1)
print("ma2=", ma2)
```



```
ma1= [None, None, None, None, 12.4, 13.0, 14.0, 15.0, 15.8, 16.4]
ma2= [ nan  nan  nan  nan 12.4 13.  14.  15.  15.8 16.4]
```

backtesting.py

- Open sourced python library for backtesting strategy
- Official Doc: <https://github.com/kernc/backtesting.py/tree/master>

A terminal window with a light gray title bar and three colored window control buttons (red, yellow, green) on the left. The terminal has a black background and displays the command `pip3 install backtesting` in green text.

```
pip3 install backtesting
```

Example

```
from backtesting import Backtest, Strategy
from backtesting.lib import crossover
from backtesting.test import SMA, GOOG

class SmaCross(Strategy):
    def init(self):
        price = self.data.Close
        self.ma1 = self.I(SMA, price, 10)
        self.ma2 = self.I(SMA, price, 20)

    def next(self):
        if crossover(self.ma1, self.ma2):
            self.buy()
        elif crossover(self.ma2, self.ma1):
            self.sell()

bt = Backtest(GOOG, SmaCross, cash=10000, commission=0.002, exclusive_orders=True)
stats = bt.run()

print(stats)

bt.plot()
```

Import CrossOver function

Retrieve GOOG stock data

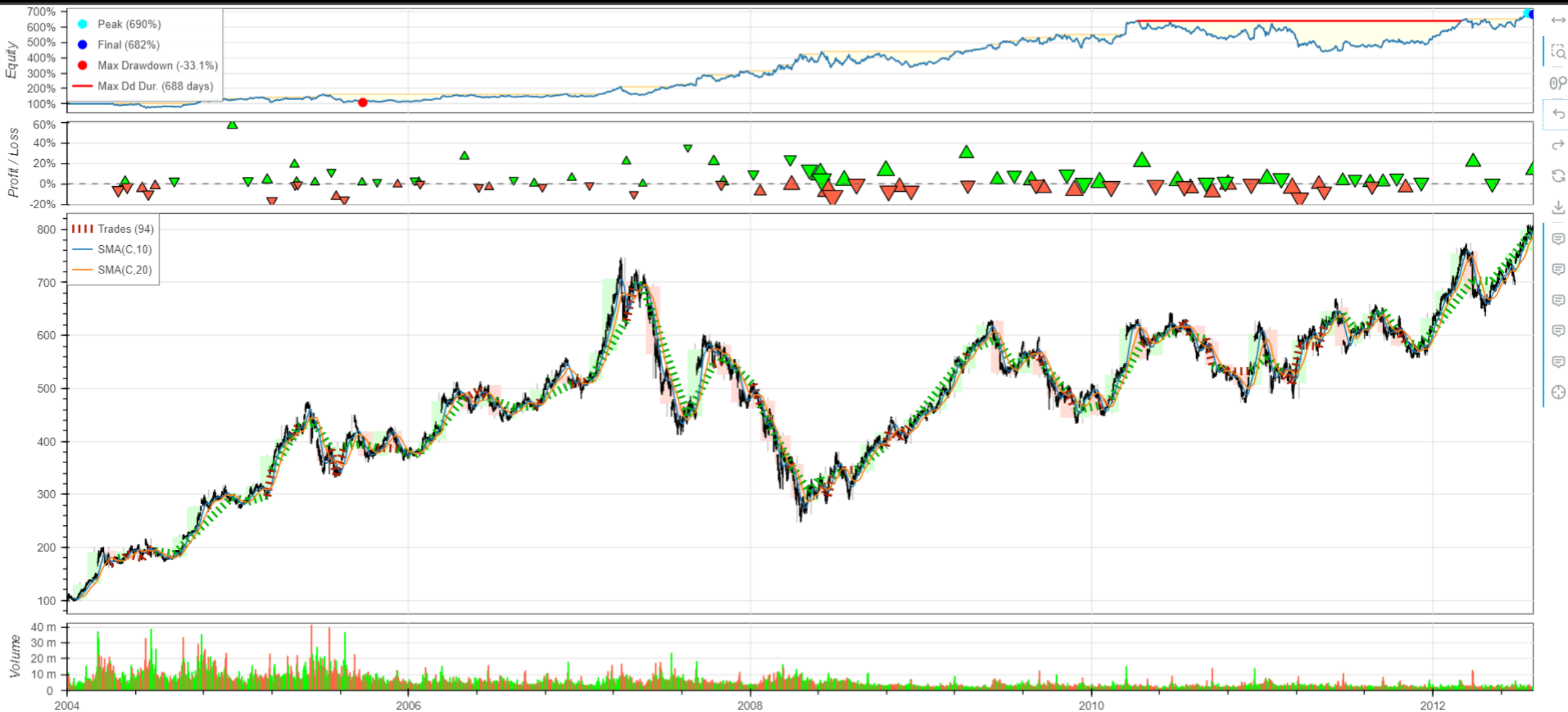
Import SMA function

Open buy when 10-day MA cross and > 20-day MA

Open sell when 20-day MA cross and > 10-day MA

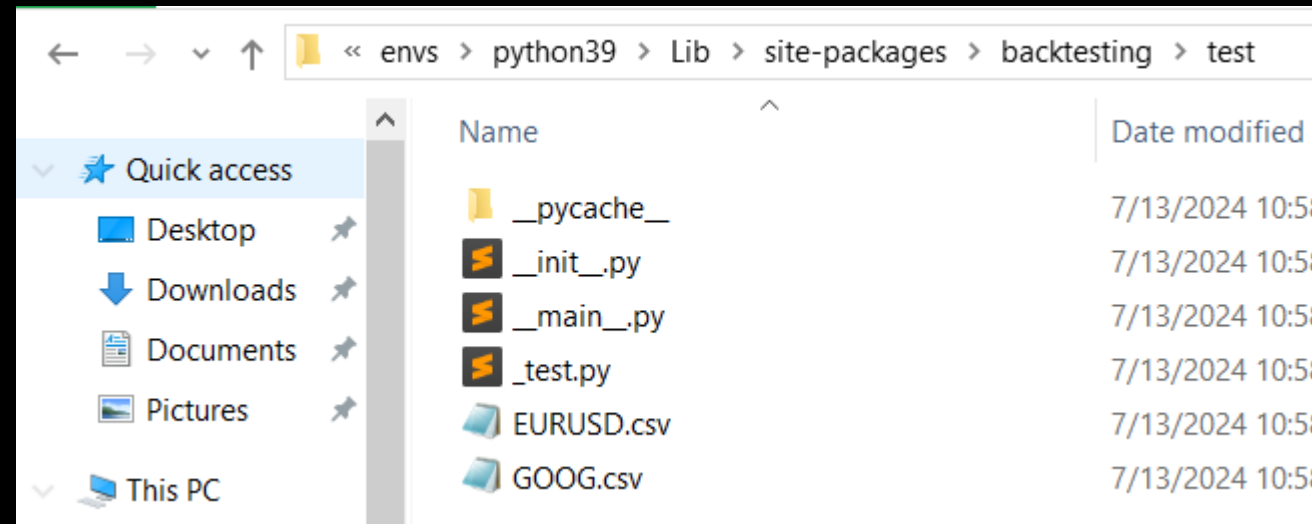
Initial cash = 10000
Commission = 0.002%
exclusive_order: True will close previous order when new signal appears

Start	2004-08-19 00:00:00
End	2013-03-01 00:00:00
Duration	3116 days 00:00:00
Exposure Time [%]	97.067039
Equity Final [\$]	68221.96986
Equity Peak [\$]	68991.21986
Return [%]	582.219699
Buy & Hold Return [%]	703.458242
Return (Ann.) [%]	25.266427
Volatility (Ann.) [%]	38.383008
Sharpe Ratio	0.658271
Sortino Ratio	1.288779
Calmar Ratio	0.763748
Max. Drawdown [%]	-33.082172
Avg. Drawdown [%]	-5.581506
Max. Drawdown Duration	688 days 00:00:00
Avg. Drawdown Duration	41 days 00:00:00
# Trades	94
Win Rate [%]	54.255319
Best Trade [%]	57.11931
Worst Trade [%]	-16.629898
Avg. Trade [%]	2.074326
Max. Trade Duration	121 days 00:00:00
Avg. Trade Duration	33 days 00:00:00
Profit Factor	2.190805
Expectancy [%]	2.606294
SQN	1.990216
_strategy	SmaCross
_equity_curve	...
_trades	Size EntryB...



Remarks

- Only **GOOG** and **EURUSD** data are preinstalled
- Need to maintain your data sources
- .csv columns need to be in the order of [Date,Open,High,Low,Close,Volume]



backtesting.py with custom data

```
from backtesting import Backtest, Strategy
from backtesting.lib import crossover
from backtesting.test import SMA #, GOOG

import yfinance as yf
from datetime import datetime
import pandas as pd

class SmaCross(Strategy):
    def init(self):
        price = self.data.Close
        self.ma1 = self.I(SMA, price, 10)
        self.ma2 = self.I(SMA, price, 20)

    def next(self):
        if crossover(self.ma1, self.ma2):
            self.buy()
        elif crossover(self.ma2, self.ma1):
            self.sell()

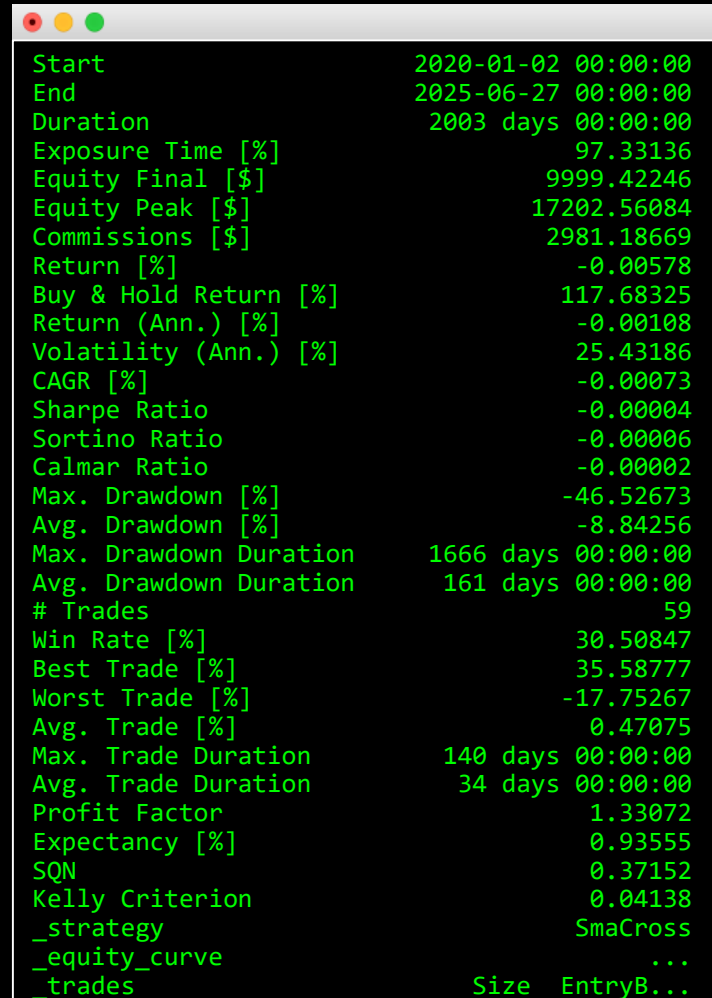
# download data from yahoo finance
tstart = datetime(2020,1,1)
tend = datetime(2025,6,30)
data = yf.download("0005.HK", start=tstart, end=tend)

# Flatten the MultiIndex columns if necessary
if isinstance(data.columns, pd.MultiIndex):
    data.columns = [col[0].strip() for col in data.columns.values]

bt = Backtest(data, SmaCross, commission=0.002, exclusive_orders=True)

stats = bt.run()
print(stats)

bt.plot()
```



```
Start                2020-01-02 00:00:00
End                  2025-06-27 00:00:00
Duration              2003 days 00:00:00
Exposure Time [%]    97.33136
Equity Final [$]     9999.42246
Equity Peak [$]      17202.56084
Commissions [$]      2981.18669
Return [%]           -0.00578
Buy & Hold Return [%] 117.68325
Return (Ann.) [%]    -0.00108
Volatility (Ann.) [%] 25.43186
CAGR [%]             -0.00073
Sharpe Ratio         -0.00004
Sortino Ratio        -0.00006
Calmar Ratio         -0.00002
Max. Drawdown [%]    -46.52673
Avg. Drawdown [%]    -8.84256
Max. Drawdown Duration 1666 days 00:00:00
Avg. Drawdown Duration 161 days 00:00:00
# Trades              59
Win Rate [%]         30.50847
Best Trade [%]       35.58777
Worst Trade [%]      -17.75267
Avg. Trade [%]       0.47075
Max. Trade Duration  140 days 00:00:00
Avg. Trade Duration  34 days 00:00:00
Profit Factor        1.33072
Expectancy [%]       0.93555
SQN                  0.37152
Kelly Criterion       0.04138
_strategy            SmaCross
_equity_curve        ...
_trades              Size EntryB...
```



backtesting.py with custom strategy

```
from backtesting import Backtest, Strategy
import yfinance as yf
from datetime import datetime
import pandas as pd

class my_MACross(Strategy):
    def init(self):
        super().init()

    def next(self):
        if self.data.signal==1:
            self.buy()
        elif self.data.signal==-1:
            self.sell()

# download data
tstart = datetime(2020,1,1)
tend = datetime(2025,6,30)
df = yf.download("0005.HK", start=tstart, end=tend)

# Flatten the MultiIndex columns if necessary
if isinstance(df.columns, pd.MultiIndex):
    df.columns = [col[0].strip() for col in df.columns.values]

# create our trading signal
df['MA_fast'] = df['Close'].rolling(7).mean()
df['MA_slow'] = df['Close'].rolling(14).mean()

# drop NA records
df.dropna(inplace=True)

# Create the signal column
df['signal'] = 0
df.loc[(df['MA_fast'] > df['MA_slow']) & (df['MA_fast'].shift(1) <= df['MA_slow'].shift(1)), 'signal'] = 1 # Golden Cross
df.loc[(df['MA_fast'] < df['MA_slow']) & (df['MA_fast'].shift(1) >= df['MA_slow'].shift(1)), 'signal'] = -1 # Death Cross

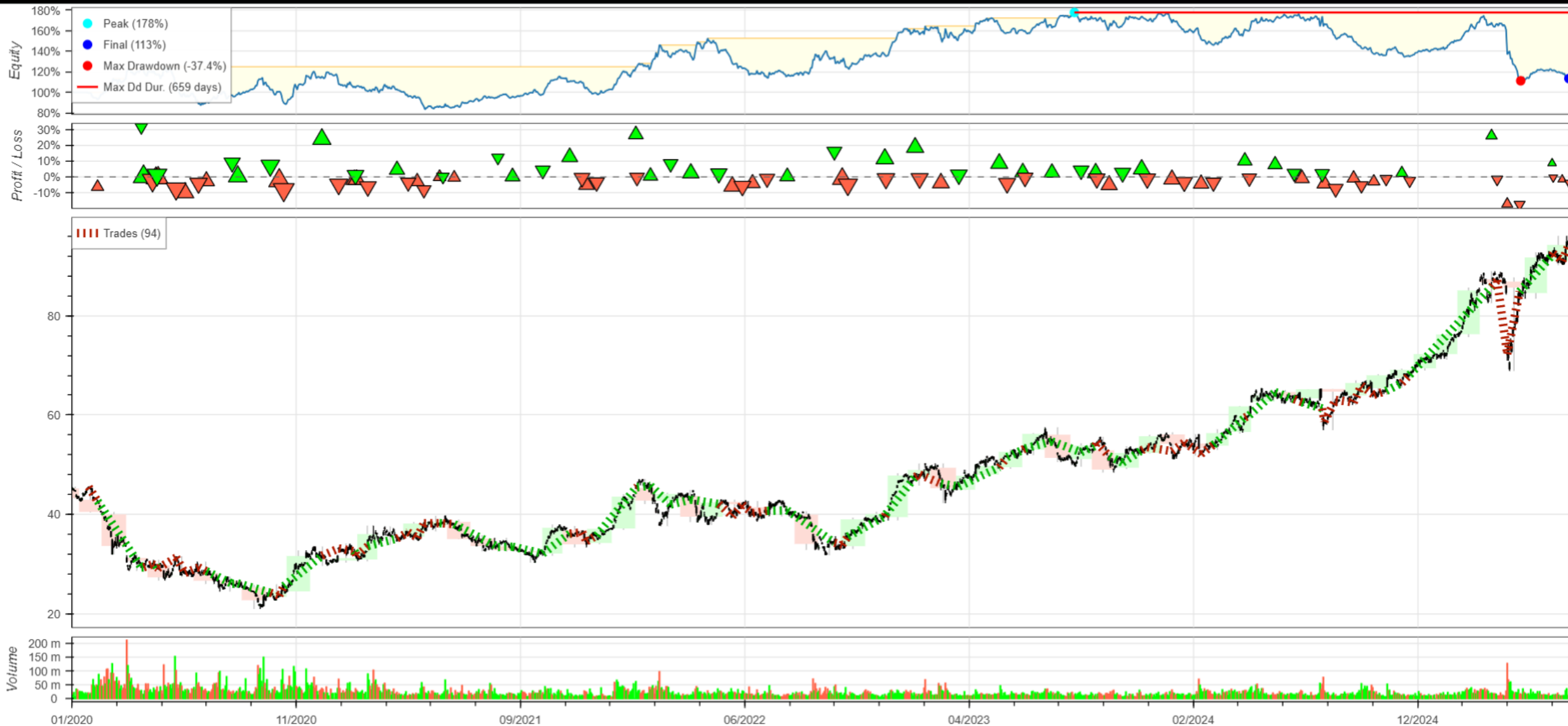
print('df=',df)

bt = Backtest(df, my_MACross, commission=0.002, exclusive_orders=True)

stats = bt.run()

print(stats)
bt.plot()
```

```
Start                2020-01-21 00:00:00
End                  2025-06-27 00:00:00
Duration              1984 days 00:00:00
Exposure Time [%]    98.87725
Equity Final [$]      11348.48556
Equity Peak [$]       17775.8496
Commissions [$]       4884.52945
Return [%]            13.48486
Buy & Hold Return [%] 109.21881
Return (Ann.) [%]     2.41476
Volatility (Ann.) [%] 26.03221
CAGR [%]              1.61972
Sharpe Ratio          0.09276
Sortino Ratio         0.13548
Calmar Ratio          0.0646
Max. Drawdown [%]    -37.38035
Avg. Drawdown [%]    -6.68342
Max. Drawdown Duration 659 days 00:00:00
Avg. Drawdown Duration 77 days 00:00:00
# Trades              94
Win Rate [%]         40.42553
Best Trade [%]       31.33929
Worst Trade [%]      -17.41497
Avg. Trade [%]        0.53888
Max. Trade Duration  111 days 00:00:00
Avg. Trade Duration   21 days 00:00:00
Profit Factor         1.3844
Expectancy [%]        0.83564
SQN                   0.67682
Kelly Criterion        0.07645
_strategy              my_MACross
_equity_curve           ...
_trades                 Size EntryB...
```

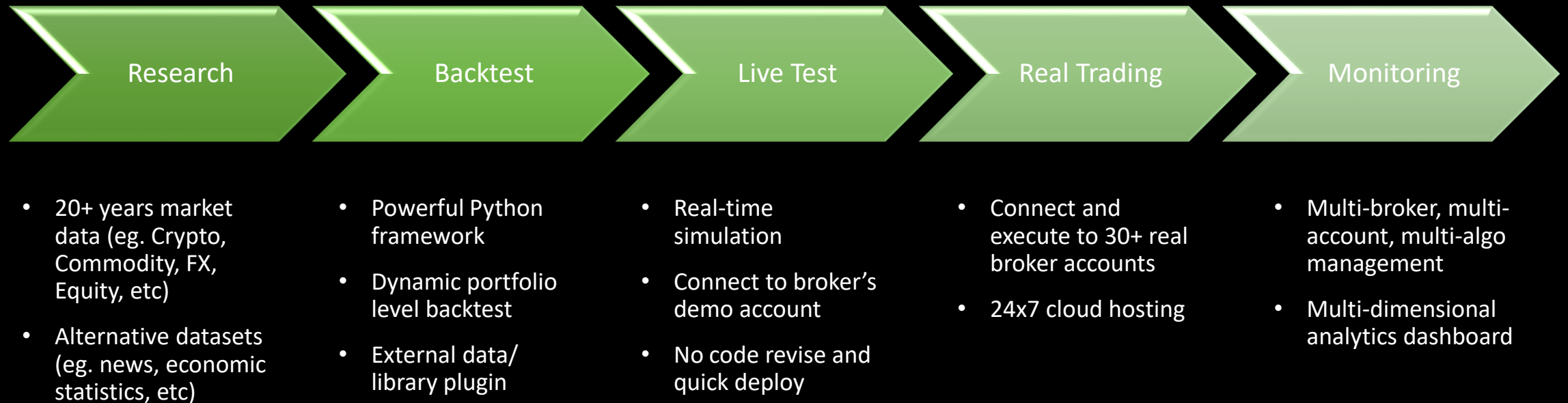


Backtest with ALGOGENE

<https://algogene.com/>

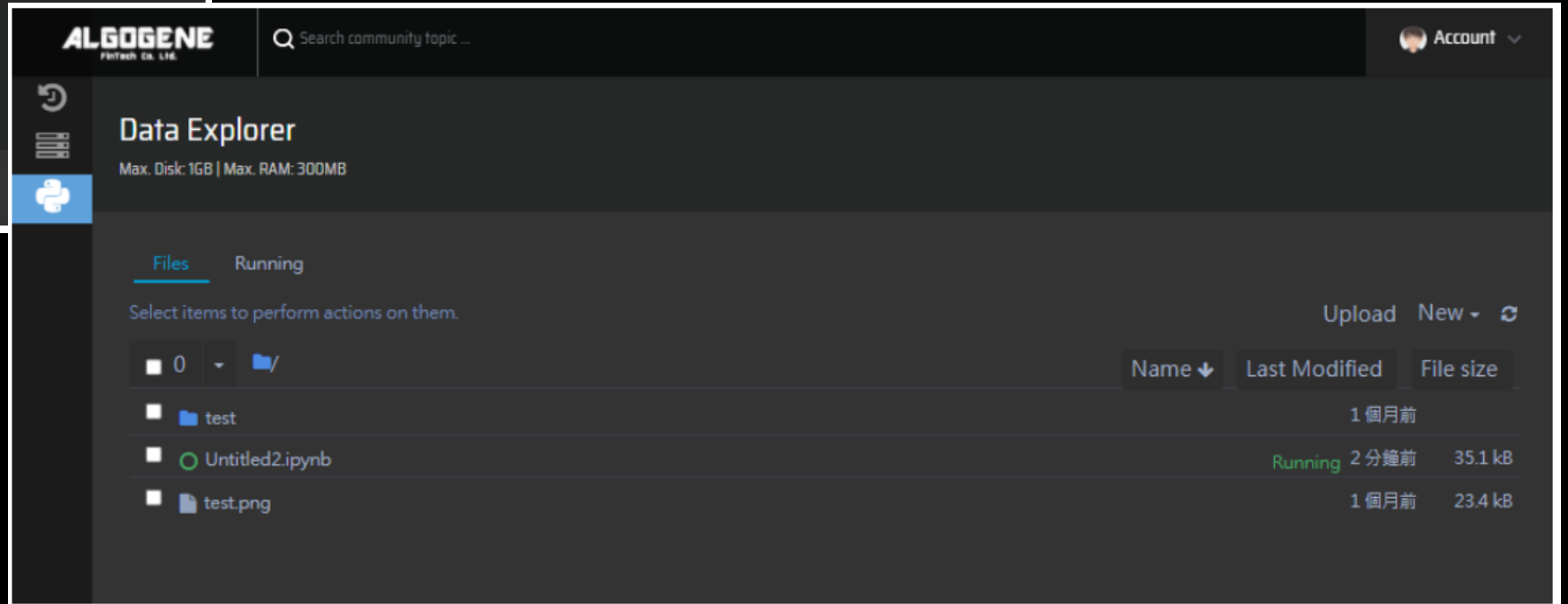
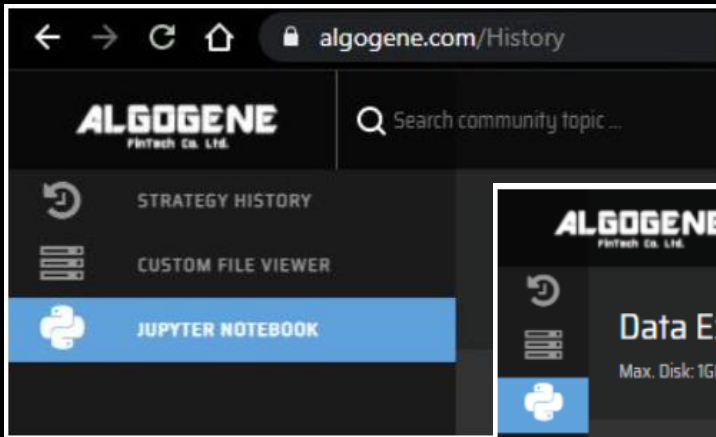
What can ALGOGENE do?

It supports the whole algo-trading life cycle



Data Exploration

- Go to [My History] > [Jupyter Notebook]



Get historical data

Data query on cloud

(<https://algotene.com/TechDoc#QueryHistory>)

```
from AlgoAPI.AlgoAPIUtil import getHistoricalBar
```

```
data = getHistoricalBar(  
    contract={"instrument": "00005HK"},  
    numofBar=100,  
    interval='D',  
    timestamp="2024-06-28"  
)
```

```
for t in data:  
    print(t, data[t])
```

```
File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3
```

```
In [1]: from AlgoAPI.AlgoAPIUtil import getHistoricalBar  
  
data = getHistoricalBar(  
    contract={"instrument": "00005HK"},  
    numofBar=100,  
    interval='D',  
    timestamp="2024-06-28"  
)  
  
for t in data:  
    print(t, data[t])
```

```
2024-02-01 00:00:00 {'t': '2024-02-01 00:00:00', 'o': 61.0, 'h': 61.7, 'l': 61.0, 'c': 61.25, 'b': 61.23, 'a': 61.27, 'm': 61.25, 'v': 321.0, 'instrument': '00005HK', 'expiry': '', 'right': '', 'strike': 0, 'symbol': '00005HK', 'bb': [], 'ab': []}  
2024-02-02 00:00:00 {'t': '2024-02-02 00:00:00', 'o': 60.45, 'h': 61.4, 'l': 60.06, 'c': 60.9, 'b': 60.88, 'a': 60.92, 'm': 60.9, 'v': 469.0, 'instrument': '00005HK', 'expiry': '', 'right': '', 'strike': 0, 'symbol': '00005HK', 'bb': [], 'ab': []}  
2024-02-05 00:00:00 {'t': '2024-02-05 00:00:00', 'o': 61.2, 'h': 61.6, 'l': 60.75, 'c': 61.355, 'b': 61.33, 'a': 61.38, 'm': 61.355, 'v': 507.0, 'instrument': '00005HK', 'expiry': '', 'right': '', 'strike': 0, 'symbol': '00005HK', 'bb': [], 'ab': []}  
2024-02-06 00:00:00 {'t': '2024-02-06 00:00:00', 'o': 61.2, 'h': 61.6, 'l': 60.475, 'c': 61.045, 'b': 61.02, 'a': 61.07, 'm': 61.045, 'v': 930.0, 'instrument': '00005HK', 'expiry': '', 'right': '', 'strike': 0, 'symbol': '00005HK', 'bb': []}
```

Format data with DataFrame

```
import pandas as pd
```

```
df = pd.DataFrame(data).T.reset_index(drop=True)  
df.drop(['strike', 'right', 'expiry'], axis=1)
```

```
print(df)
```

```
File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3
```

```
In [2]: import pandas as pd  
  
df = pd.DataFrame(data).T.reset_index(drop=True)  
df = df.drop(['strike', 'right', 'expiry'], axis=1)  
  
print(df)
```

	t	o	h	l	c	b	a	m	\
0	2024-02-01 00:00:00	61.0	61.7	61.0	61.25	61.23	61.27	61.25	
1	2024-02-02 00:00:00	60.45	61.4	60.2	60.9	60.88	60.92	60.9	
2	2024-02-05 00:00:00	61.2	61.6	60.75	61.355	61.33	61.38	61.355	
3	2024-02-06 00:00:00	61.2	61.6	60.475	61.045	61.02	61.07	61.045	
4	2024-02-07 00:00:00	61.0	61.825	60.2	61.705	61.68	61.73	61.705	
..	...	...	...	...	...	...	...	...	
95	2024-06-24 00:00:00	68.55	68.775	67.975	68.15	68.15	68.15	68.15	
96	2024-06-25 00:00:00	68.55	68.775	67.225	67.85	67.82	67.88	67.85	
97	2024-06-26 00:00:00	68.2	68.575	67.95	68.4	68.4	68.4	68.4	
98	2024-06-27 00:00:00	69.5	69.5	67.5	68.3	68.27	68.33	68.3	
99	2024-06-28 00:00:00	64.75	68.625	64.75	68.3	68.27	68.33	68.3	

Get historical data via REST API

- Go to [Settings] > [Profile]
- Get your [User ID]
- [Generate] a REST API Key

ALGOGENE
FinTech Co. Ltd.

Profile

My Wallet

My Orders

Home

Services

Marketplace

Community

Account

System Info

User ID:

User Name:

Password *:

Preferred Language:

English

Referral Link: ?

Copy

REST API Key: ?

Generate

Telegram Chat ID: ?

Generate

f35dc8e7

Get historical data via REST API

https://allogene.com/RestDoc#/operations/get-query_marketprice

```
import requests
```

```
url = "https://algogene.com/rest/v1/history_price"
```

```
params = {
    "user": "XXX",
    "api_key": "XXX",
    "count": "100",
    "interval": "D",
    "instrument": "00005HK",
    "timestamp": "2024-06-28"
}
```

```
headers = {"Content-Type": "application/json"}
```

```
response = requests.request("GET", url, headers=headers, params=params)
print(response.text)
```

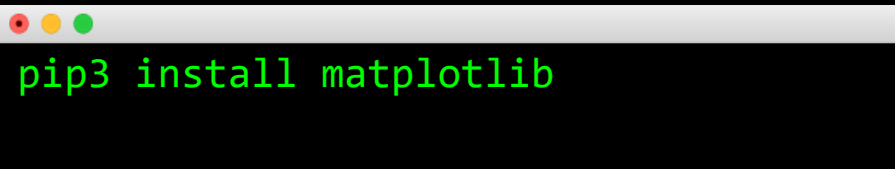
```

{"count":100,"res":[{"a":60.726463,"ab":[],"b":60.686818,"bb":[],"c":60.70664,"expiry":"","h":61.152648,"instrument":"00005HK","l":60.458858,"m":60.70664,"o":60.458858,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-01 00:00:00","v":321.0},{a":60.379568,"ab":[],"b":60.339923,"bb":[],"c":60.359745,"expiry":"","h":60.85531,"instrument":"00005HK","l":59.665955,"m":60.359745,"o":59.913737,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-02 00:00:00","v":469.0},{a":60.835487,"ab":[],"b":60.78593,"bb":[],"c":60.810709,"expiry":"","h":61.053535,"instrument":"00005HK","l":60.211076,"m":60.810709,"o":60.657084,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-05 00:00:00","v":507.0},{a":60.528237,"ab":[],"b":60.478681,"bb":[],"c":60.503459,"expiry":"","h":61.053535,"instrument":"00005HK","l":59.938515,"m":60.503459,"o":60.657084,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-06 00:00:00","v":930.0},{a":61.182382,"ab":[],"b":61.132826,"bb":[],"c":61.157604,"expiry":"","h":61.276539,"instrument":"00005HK","l":59.665955,"m":61.157604,"o":60.458858,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-07 00:00:00","v":447.0},{a":61.618479,"ab":[],"b":61.568922,"bb":[],"c":61.5937,"expiry":"","h":61.945551,"instrument":"00005HK","l":61.202205,"m":61.5937,"o":61.202205,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-08 00:00:00","v":460.0},{a":61.122914,"ab":[],"b":61.073358,"bb":[],"c":61.098136,"expiry":"","h":61.623434,"instrument":"00005HK","l":60.805753,"m":61.098136,"o":61.5491,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-09 00:00:00","v":461.0},{a":60.776019,"ab":[],"b":60.726463,"bb":[],"c":60.751241,"expiry":"","h":61.449987,"instrument":"00005HK","l":59.219947,"m":60.751241,"o":61.449987,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-14 00:00:00","v":204.0},{a":60.280455,"ab":[],"b":60.230898,"bb":[],"c":60.255677,"expiry":"","h":61.449987,"instrument":"00005HK","l":58.972165,"m":60.255677,"o":61.449987,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-15 00:00:00","v":521.0},{a":60.835487,"ab":[],"b":60.78593,"bb":[],"c":60.810709,"expiry":"","h":61.028757,"instrument":"00005HK","l":59.963294,"m":60.810709,"o":60.062406,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-16 00:00:00","v":369.0},{a":61.430164,"ab":[],"b":61.380608,"bb":[],"c":61.405386,"expiry":"","h":62.193333,"instrument":"00005HK","l":59.963294,"m":61.405386,"o":60.954422,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-19 00:00:00","v":503.0},{a":61.846438,"ab":[],"b":61.846438,"bb":[],"c":61.846438,"expiry":"","h":62.193333,"instrument":"00005HK","l":59.963294,"m":61.846438,"o":60.954422,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-20 00:00:00","v":1127.0},{a":62.114043,"ab":[],"b":62.064487,"bb":[],"c":62.089265,"expiry":"","h":62.267668,"instrument":"00005HK","l":59.963294,"m":62.089265,"o":62.24289,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-21 00:00:00","v":344.0},{a":59.715511,"ab":[],"b":59.715511,"bb":[],"c":59.715511,"expiry":"","h":62.892079,"instrument":"00005HK","l":59.51233,"m":59.715511,"o":62.441116,"right":"","strike":0,"symbol":"00005HK","t":"2024-02-22 00:00:00","v":776.0},{a":59.0911,"ab":[],"b":59.051455,"bb":[],"c":59.071278,"expiry":"","h":59.418173,"in

```

Visualize Data with Matplotlib

- Matplotlib is a comprehensive Python library for creating static, animated, and interactive charts.
- It is highly customizable and integrates well with other libraries like NumPy and Pandas.
- Official Doc: <https://matplotlib.org/stable/tutorials/pyplot.html>

A terminal window with a light gray title bar and three colored window control buttons (red, yellow, green) on the left. The terminal has a black background and displays the command `pip3 install matplotlib` in green text.

```
pip3 install matplotlib
```

Create Line Plots

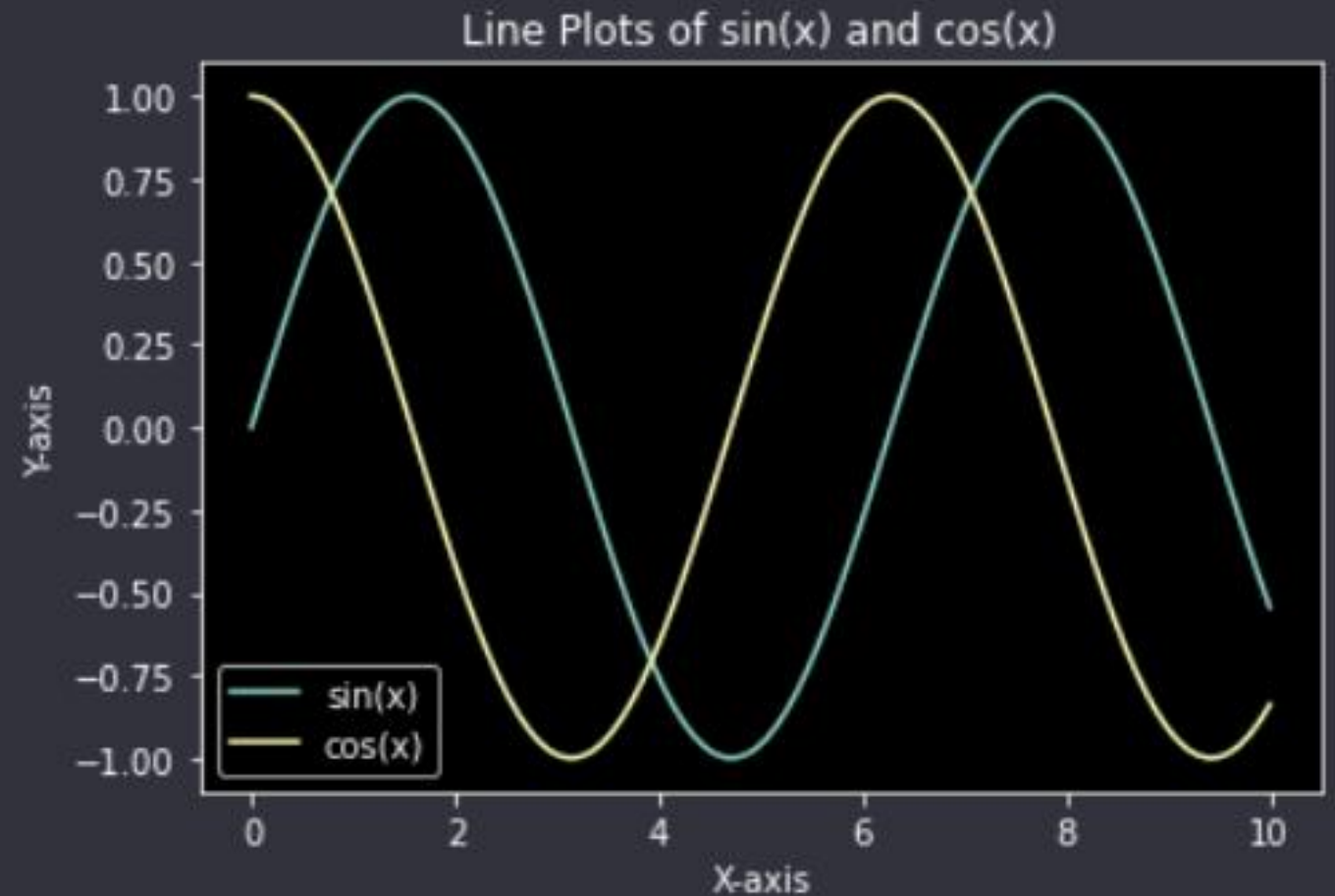
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, label='sin(x)')
plt.plot(x, y2, label='cos(x)')

plt.title('Line Plots of sin(x) and cos(x)')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')

plt.legend()
plt.show()
```



Visualize Stock Data

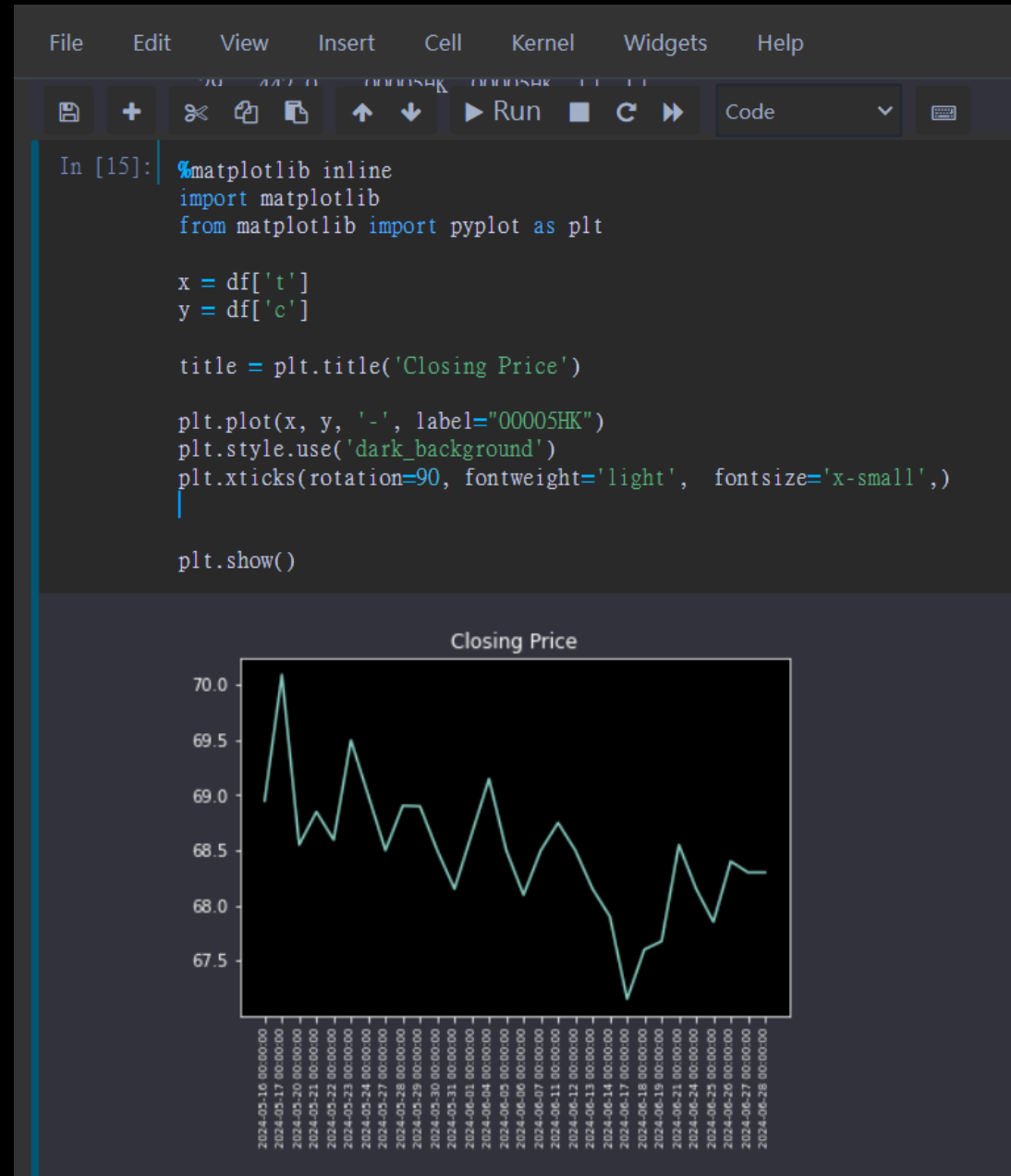
```
%matplotlib inline import matplotlib
from matplotlib import pyplot as plt
```

```
x = df['t']
y = df['c']
```

```
title = plt.title('Closing Price')
```

```
plt.plot(x, y, '-', label="00005HK")
plt.style.use('dark_background')
plt.xticks(rotation=90, fontweight='light', fontsize='x-small',)
```

```
plt.show()
```



Backtest Interface

4. Backtest results

2. Backtest setting

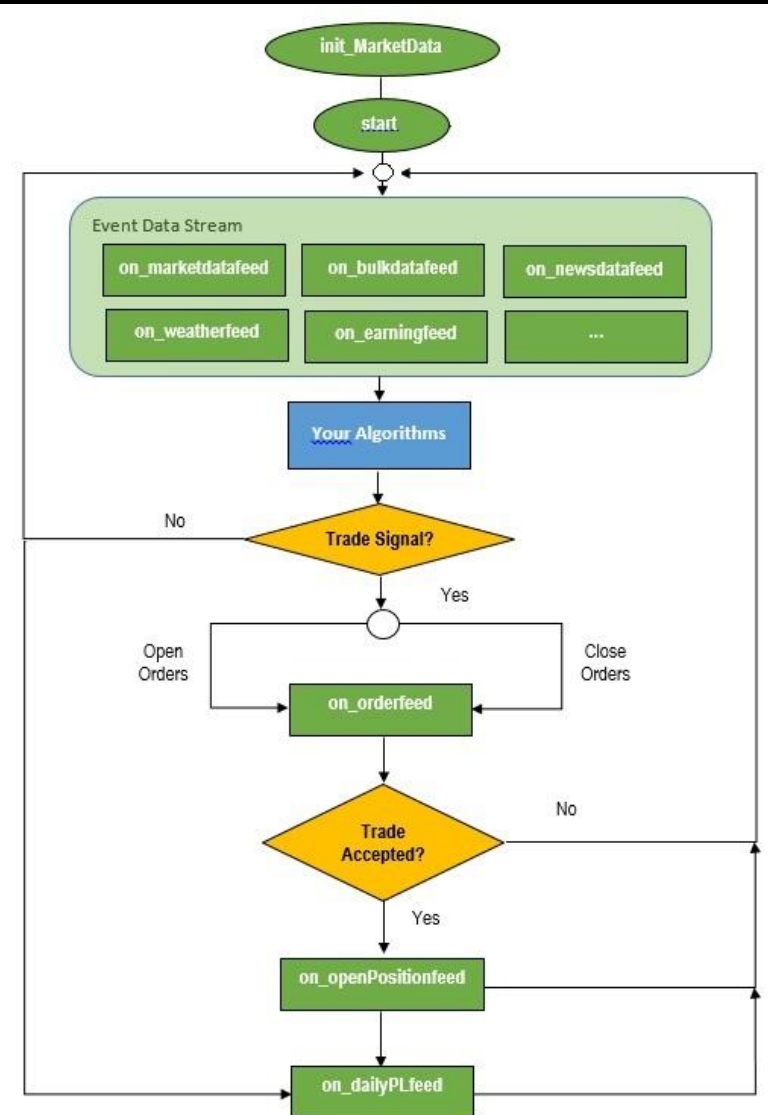
3. Technical Doc

The screenshot displays the Backtest Interface with the following components:

- Top Navigation Bar:** Contains tabs for Script, Report, Chart, Market_Price, PnL, Position, Account_Balance, Cash_Flow, Trade_Details, and Console. The Script tab is highlighted with a red box.
- Buttons:** Run (green), Stop (red), and Reset (blue) buttons are located below the navigation bar.
- Script Editor:** Displays Python code for the AlgoEvent class, including methods for __init__, start, and various data feeds (bulk, market, news, weather, econs, corpAnnouncement, order, dailyPL, openPosition). A red arrow points to the code area, labeled "1. Coding".
- Settings Panel:** A panel on the right side titled "Settings" containing various configuration options:
 - Strategy Name: Text input field.
 - Start Period *: Dropdown menu (2021-01).
 - End Period *: Dropdown menu (2021-01).
 - Data Interval *: Dropdown menu (--- Select Data Interval ---).
 - Initial Capital *: Text input field (100000).
 - Base Currency *: Dropdown menu (AUD).
 - Leverage: Text input field (1).
 - Transaction Cost: Text input field (0).
 - Risk free Rate: Text input field (0).
 - Enable Short Sell: Toggle switch (on).
 - Position Netting: Toggle switch (on).
 - News Data: Toggle switch (on).
 - Weather Data: Toggle switch (on).
 - Economics Data: Toggle switch (on).
 - Instruments: Dropdown menu (Select ...).
- Tech Doc:** A link labeled "Tech Doc" is located in the top right corner, with a red arrow pointing to it, labeled "3. Technical Doc".

Backtest Framework

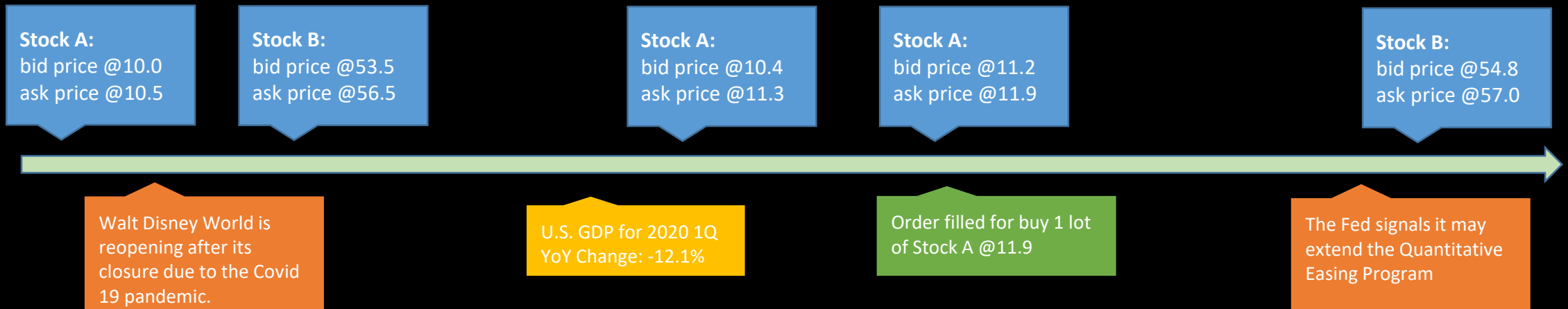
```
1 from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
2
3 class AlgoEvent:
4     def __init__(self):
5         pass
6
7     def start(self, mEvt):
8         self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
9         self.evt.start()
10
11     def on_bulkdatafeed(self, isSync, bd, ab):
12         pass
13
14     def on_marketdatafeed(self, md, ab):
15         pass
16
17     def on_newsdatafeed(self, nd):
18         pass
19
20     def on_weatherdatafeed(self, wd):
21         pass
22
23     def on_econsdatafeed(self, ed):
24         pass
25
26     def on_corpAnnouncement(self, ca):
27         pass
28
29     def on_orderfeed(self, of):
30         pass
31
32     def on_dailyPLfeed(self, pl):
33         pass
34
35     def on_openPositionfeed(self, op, oo, uo):
36         pass
37
```



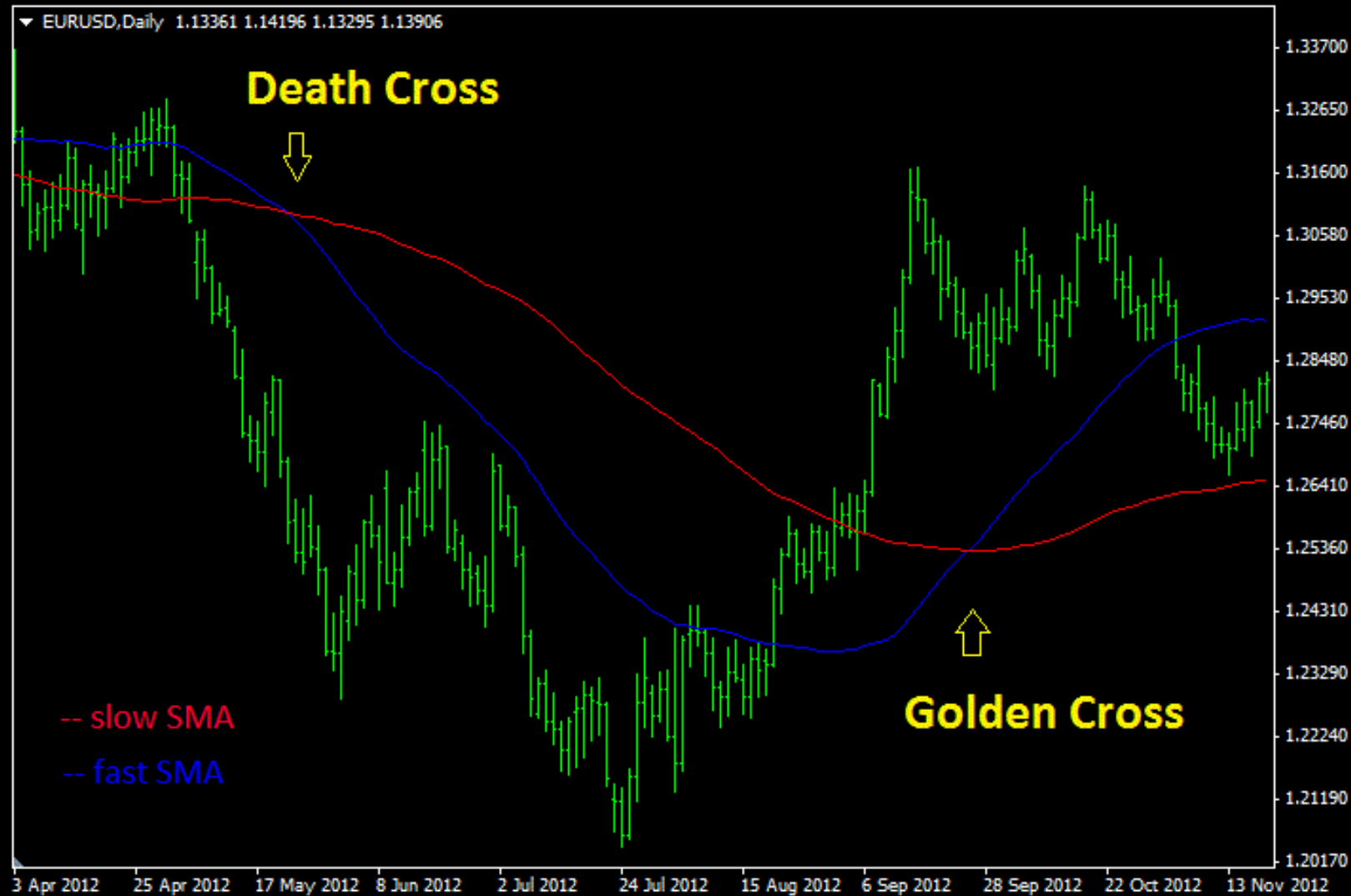
How data is processed on ALGOGENE?

- Event Stream Architecture

- Data feed
 - Market data
 - News
 - Economic statistics
- Order status update
- Position changes
- ...



MA Cross Strategy



MA Cross Strategy

Backtest setup

- Data interval = 1 day
- Base currency = HKD
- Leverage = 1x
- Enable short sell = False
- Instrument = 00005HK

Settings

Strategy Name

TEST

Start Period *

2024-01

End Period *

2024-12

Data Interval *

1 day

Initial Capital *

100000

Base Currency *

HKD

Leverage

1

Transaction Cost

0

Risk free Rate

0

Enable Short Sell

☐

Position Netting

☐

News Data

☐

Weather Data

☐

Economics Data

☐

Instruments ⓘ
(Max. 100 items)

Select ...

00005HK ×

MA Cross Strategy

1. Initialize variable

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
from datetime import datetime, timedelta
import talib, numpy
```

← Import library

```
class AlgoEvent:
    def __init__(self):
        self.last_time = datetime(1970,1,1)
        self.arr_close = numpy.array([])
        self.arr_fastMA = numpy.array([])
        self.arr_slowMA = numpy.array([])
        self.fastperiod = 7
        self.slowperiod = 14
```

← Define our variables under *def __init__*

MA Cross Strategy

2. Calculate MA value

```
def on_marketdatafeed(self, md, ab):
    # retrieve recent observations
    res = self.evt.getHistoricalBar(
        contract = {"instrument": md.instrument},
        numOfBar = self.fastperiod+self.slowperiod,
        interval = "D"
    )

    # get closing price data
    self.arr_close = numpy.array([res[t]['c'] for t in res])

    # fit SMA line
    self.arr_fastMA = talib.SMA(self.arr_close, timeperiod=int(self.fastperiod))
    self.arr_slowMA = talib.SMA(self.arr_close, timeperiod=int(self.slowperiod))

    # debug print result
    self.evt.consoleLog("arr_fastMA=", self.arr_fastMA)
    self.evt.consoleLog("arr_slowMA=", self.arr_slowMA)
```

```
[2023-07-01 00:00:00] Backtest started!
[2023-07-01 00:00:00] arr_fastMA=, [ nan nan nan nan nan nan
59.05714286 59.43571429 59.67857143 59.80714286 59.95 60.13571429
60.34285714 60.55714286 60.74285714 60.71428571 60.65 60.71428571
60.69285714 60.61428571 60.54285714]
[2023-07-01 00:00:00] arr_slowMA=, [ nan nan nan nan nan nan
nan nan nan nan nan nan
nan 59.80714286 60.08928571 60.19642857 60.22857143 60.33214286
60.41428571 60.47857143 60.55 ]
[2023-07-04 00:00:00] arr_fastMA=, [ nan nan nan nan nan nan
59.43571429 59.67857143 59.80714286 59.95 60.13571429 60.34285714
60.55714286 60.74285714 60.71428571 60.65 60.71428571 60.69285714
60.61428571 60.54285714 60.63571429]
[2023-07-04 00:00:00] arr_slowMA=, [ nan nan nan nan nan nan
nan nan nan nan nan nan
nan 60.08928571 60.19642857 60.22857143 60.33214286 60.41428571
60.47857143 60.55 60.68928571]
[2023-07-05 00:00:00] arr_fastMA=, [ nan nan nan nan nan nan
59.67857143 59.80714286 59.95 60.13571429 60.34285714 60.55714286
60.74285714 60.71428571 60.65 60.71428571 60.69285714 60.61428571
60.54285714 60.63571429 61. ]
[2023-07-05 00:00:00] arr_slowMA=, [ nan nan nan nan nan nan
nan nan nan nan nan nan
nan 60.19642857 60.22857143 60.33214286 60.41428571 60.47857143
60.55 60.68928571 60.85714286]
```

MA Cross Strategy

3. Query order inventory

<https://algotene.com/TechDoc#QueryServerOrder>

```
def on_marketdatafeed(self, md, ab):
    # retrieve recent observations
    res = self.evt.getHistoricalBar(
        contract = {"instrument": md.instrument},
        numofBar = self.fastperiod+self.slowperiod,
        interval = "D"
    )

    # get closing price data
    self.arr_close = numpy.array([res[t]['c'] for t in res])

    # fit SMA line
    self.arr_fastMA = talib.SMA(self.arr_close, timeperiod=int(self.fastperiod))
    self.arr_slowMA = talib.SMA(self.arr_close, timeperiod=int(self.slowperiod))

    # debug print result
    self.evt.consoleLog("arr_fastMA=", self.arr_fastMA)
    self.evt.consoleLog("arr_slowMA=", self.arr_slowMA)

    # check number of record is at least greater than both self.fastperiod, self.slowperiod
    if numpy.isnan(self.arr_fastMA[-1]) or numpy.isnan(self.arr_fastMA[-2]) or \
        numpy.isnan(self.arr_slowMA[-1]) or numpy.isnan(self.arr_slowMA[-2]):
        return

    # query outstanding position
    positions, osOrder, pendOrder = self.evt.getSystemOrders()
    pos = positions[md.instrument]['netVolume']
```

MA Cross Strategy

4. Order Entry Condition

Open a buy order for golden cross

```
def on_marketdatafeed(self, md, ab):
    # retrieve recent observations
    res = self.evt.getHistoricalBar(
        contract = {"instrument": md.instrument},
        numOfBar = self.fastperiod+self.slowperiod,
        interval = "D"
    )

    # get closing price data
    self.arr_close = numpy.array([res[t]['c'] for t in res])

    # fit SMA line
    self.arr_fastMA = talib.SMA(self.arr_close, timeperiod=int(self.fastperiod))
    self.arr_slowMA = talib.SMA(self.arr_close, timeperiod=int(self.slowperiod))

    # debug print result
    self.evt.consoleLog("arr_fastMA=", self.arr_fastMA)
    self.evt.consoleLog("arr_slowMA=", self.arr_slowMA)

    # check number of record is at least greater than both self.fastperiod, self.slowperiod
    if numpy.isnan(self.arr_fastMA[-1]) or numpy.isnan(self.arr_fastMA[-2]) or \
        numpy.isnan(self.arr_slowMA[-1]) or numpy.isnan(self.arr_slowMA[-2]):
        return

    # query outstanding position
    positions, osOrder, pendOrder = self.evt.getSystemOrders()
    pos = positions[md.instrument]["netVolume"]

    # send a buy order for Golden Cross
    if self.arr_fastMA[-1] > self.arr_slowMA[-1] and self.arr_fastMA[-2] < self.arr_slowMA[-2]:
        order = AlgoAPIUtil.OrderObject(
            instrument = md.instrument,
            buysell = 1,
            volume = 1,
            openclose = "open",
            ordertype = 0 #0=market_order, 1=limit_order, 2=stop_order
        )
        self.evt.sendOrder(order)
```

MA Cross Strategy

5. Order Exit Condition

Close previous buy order when death cross appears

```
# query outstanding position
positions, osOrder, pendOrder = self.evt.getSystemOrders()
pos = positions[md.instrument]['netVolume']

# send a buy order for Golden Cross
if self.arr_fastMA[-1] > self.arr_slowMA[-1] and self.arr_fastMA[-2] < self.arr_slowMA[-2]:
    order = AlgoAPIUtil.OrderObject(
        instrument = md.instrument,
        buySell = 1,
        volume = 1,
        openClose = "open",
        ordertype = 0 #0=market_order, 1=limit_order, 2=stop_order
    )
    self.evt.sendOrder(order)

# send a sell order for Death Cross
if self.arr_fastMA[-1] < self.arr_slowMA[-1] and self.arr_fastMA[-2] > self.arr_slowMA[-2]:
    # close all outstanding position
    for tradeID in list(osOrder):
        order = AlgoAPIUtil.OrderObject(
            tradeID = tradeID,
            openClose = "close"
        )
        self.evt.sendOrder(order)
```

MA Cross Strategy

Full script

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
from datetime import datetime, timedelta
import talib, numpy

class AlgoEvent:
    def __init__(self):
        self.last_day = datetime(1970,1,1)
        self.arr_close = numpy.array([])
        self.arr_fastMA = numpy.array([])
        self.arr_slowMA = numpy.array([])
        self.fastperiod = 7
        self.slowperiod = 14

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()

    def on_marketdatafeed(self, md, ab):
        # retrieve recent observations
        res = self.evt.getHistoricalBar(
            contract = {"instrument": md.instrument},
            numOfBar = self.fastperiod+self.slowperiod,
            interval = "D"
        )

        # get closing price data
        self.arr_close = numpy.array([res[t][c] for t in res])

        # fit SMA line
        self.arr_fastMA = talib.SMA(self.arr_close, timeperiod=int(self.fastperiod))
        self.arr_slowMA = talib.SMA(self.arr_close, timeperiod=int(self.slowperiod))

        # debug print result
        self.evt.consoleLog("arr_fastMA=", self.arr_fastMA)
        self.evt.consoleLog("arr_slowMA=", self.arr_slowMA)

        # check number of record is at least greater than both self.fastperiod, self.slowperiod
        if numpy.isnan(self.arr_fastMA[-1]) or numpy.isnan(self.arr_fastMA[-2]) or numpy.isnan(self.arr_slowMA[-1]) or numpy.isnan(self.arr_slowMA[-2]):
            return

        # query outstanding position
        positions, osOrder, pendOrder = self.evt.getSystemOrders()
        pos = positions[md.instrument]["netVolume"]

        # send a buy order for Golden Cross
        if self.arr_fastMA[-1] > self.arr_slowMA[-1] and self.arr_fastMA[-2] < self.arr_slowMA[-2]:
            order = AlgoAPIUtil.OrderObject(
                instrument = md.instrument,
                buysell = 1,
                volume = 1,
                openclose = "open",
                ordertype = 0 #0=market_order, 1=limit_order, 2=stop_order
            )
            self.evt.sendOrder(order)

        # send a sell order for Death Cross
        if self.arr_fastMA[-1] < self.arr_slowMA[-1] and self.arr_fastMA[-2] > self.arr_slowMA[-2]:
            # close all outstanding position
            for tradeID in osOrder:
                order = AlgoAPIUtil.OrderObject(
                    tradeID = tradeID,
                    openclose = "close"
                )
                self.evt.sendOrder(order)
```

MA Cross Strategy

No. of Tradable Days:	248	No. of Win Days:	76	No. of Loss Days:	61
Win Rate:	55.4745%	Max. Consecutive Win Day:	7	Max. Consecutive Loss Day:	3
Odd Ratio:	1.2459	Max. Consecutive Gains:	2368.0	Max. Consecutive Loss:	-2484.0
No. of Trades:	25	Average Consecutive Win Day:	0.92	Average Consecutive Loss Day:	0.41
Total PnL:	1238.4	Average Per Trade PnL:	49.54	Average Per Day PnL:	4.99
Mean Daily Return:	0.006%	Median Daily Return:	0.0%	Mean Annual Return:	1.5048%
Daily Return StdDev:	0.3312%	25th percentile Daily Return:	-0.02%	75th percentile Daily Return:	0.0937%
Daily Return Downside StdDev:	0.2367%	95% 1 day return VaR:	-0.3948%	99% 1 day return VaR:	-1.0166%
Daily Sharpe Ratio:	0.018	Annual Sharpe Ratio:	0.2862	Max. Drawdown Amount:	5376.0
Daily Sortino Ratio:	0.0252	Annual Sortino Ratio:	0.4004	Max. Drawdown Percent:	5.24%
Max. Drawdown Duration:	159	Average Drawdown Duration:	58.58	Annual Volatility:	5.2363%
Gross Profit:	3500.0	Gross Loss:	-4680.0	Profit Factor:	0.7479
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treynor Ratio:	0.0	Tail Ratio:	0.9591
Calmar Ratio:	0.2872	Average Holding Day:	14.9231	Annual Turnover Rate:	319.708%

Wrap Up

	Pros	Cons
Excel	• Quick to test simple ideas • Easy to create charts and adjust formulas	• Need to manage your data sources • Lacks of feasibility to scale and extend to complicated strategies • Unable to load large amount of data into spreadsheet
Backtesting.py	• Free and simple python code • Can extend to other open-sourced libraries for different technical indicators, statistical models, etc	• Need to manage your data sources • Limited build-in functionalities • Lack of supports for real trading deployment • Limited memory to load large amount of data into dataframe
AlgoGene	• Python environment with open-sourced libraries available • Ready-to-use datasets • Seamless deployment from backtest to live test, real trading without code amendment	• May need some time to get familiar with the system

Common Backtest Pitfalls

Common Pitfalls

1. Overfitting
2. Look-ahead bias
3. Survivorship Bias
4. Data-Snooping Bias
5. Ignore assumptions you made

Overfitting

- **Definition:** Tailoring a strategy too closely to historical data, making it less likely to perform well on new, unseen data.
- **Symptoms:** Exceptional past performance without sound reasoning.
- **Avoidance:** Use out-of-sample testing and cross-validation.

Look-ahead bias

- **Definition:** Using future data that would not have been available at the time of the trade decision.
- **Symptoms:** Unrealistically high profits.
- **Avoidance:** Ensure all data used for decisions is available up to the point in time being simulated.

Survivorship Bias

- **Definition:** Only considering assets that have survived until the end of the period, ignoring those that have failed.
- **Symptoms:** Overestimation of strategy performance.
- **Avoidance:** Include delisted or bankrupt companies in the dataset.

Data-Snooping Bias

- **Definition:** Repeatedly testing multiple strategies on the same dataset until one works, which may be due to chance.
- **Symptoms:** High chance of false positives.
- **Avoidance:** Validate the strategy on different datasets or time periods.

Ignore Assumptions

- **Examples:**

- Failing to account for costs such as commissions, slippage, financing costs, and taxes
- Short selling on certain stocks cannot be traded in real market
- Exceptionally high leverage is not doable in practice
- Potential delay between signal calculation and execution (eg. during market closure, using a complicated trading model, etc)

- **Avoidance:**

- Incorporate realistic estimates of transaction costs into the backtest.
- Do some market research to check if your assumptions can be implemented in real market

Best Practice

- **Robust Validation:** Use out-of-sample testing and walk-forward analysis.
- **Realistic Assumptions:** Incorporate transaction costs and market impact.
- **Continuous Monitoring:** Regularly update and review the strategy.

Exercise

RSI Strategy

Relative Strength Index (RSI)

- a momentum oscillator that measures the speed and change of price movements
- typically used to identify overbought or oversold conditions in a market
- It ranges from 0 to 100
 - Overbought: $RSI > 70$, indicating that the asset may be overvalued and a price correction could occur.
 - Oversold: $RSI < 30$, suggesting that the asset may be undervalued and a price increase could occur.

Relative Strength Index (RSI)

Define:

- **Average Gain:** Sum of gains over a specified period divided by the number of periods.
- **Average Loss:** Sum of losses over the same period divided by the number of periods.
- **Period:** Commonly 14 days.

Formula:

$$\text{RS} = \text{Average Gain} / \text{Average Loss}$$

$$\text{RSI} = 100 - (100 / (1 + \text{RS}))$$

Example

- Calculate RSI(7) for the following daily stock price

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Price	44.34	44.09	44.15	43.61	44.33	45.32	45.93	45.45	46.25	46.92	47.25	46.8	46	46.25	46.78	47.1	46.9	47.45	48.1	48.5

- Calculate the daily gain and loss

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Price	44.34	44.09	44.15	43.61	44.33	45.32	45.93	45.45	46.25	46.92	47.25	46.8	46	46.25	46.78	47.1	46.9	47.45	48.1	48.5
Diff		-0.25	0.06	-0.54	0.72	0.99	0.61	-0.48	0.8	0.67	0.33	-0.45	-0.8	0.25	0.53	0.32	-0.2	0.55	0.65	0.4
Gain		0	0.06	0	0.72	0.99	0.61	0	0.8	0.67	0.33	0	0	0.25	0.53	0.32	0	0.55	0.65	0.4
Loss		0.25	0	0.54	0	0	0	0.48	0	0	0	0.45	0.8	0	0	0	0.2	0	0	0
Avg Gain								0.34	0.454286	0.541429	0.588571	0.485714	0.344286	0.292857	0.368571	0.3	0.204286	0.235714	0.328571	0.385714
Avg Loss								0.181429	0.145714	0.145714	0.068571	0.132857	0.247143	0.247143	0.178571	0.178571	0.207143	0.207143	0.142857	0.028571
RS								1.874016	3.117647	3.715686	8.583333	3.655914	1.393064	1.184971	2.064	1.68	0.986207	1.137931	2.3	13.5
RSI								65.20548	75.71429	78.79418	89.56522	78.52194	58.21256	54.2328	67.36292	62.68657	49.65278	53.22581	69.69697	93.10345

Example

```
import pandas as pd

def calculate_rsi(data, window=7):
    """
    Calculate the Relative Strength Index (RSI) for a given dataset.

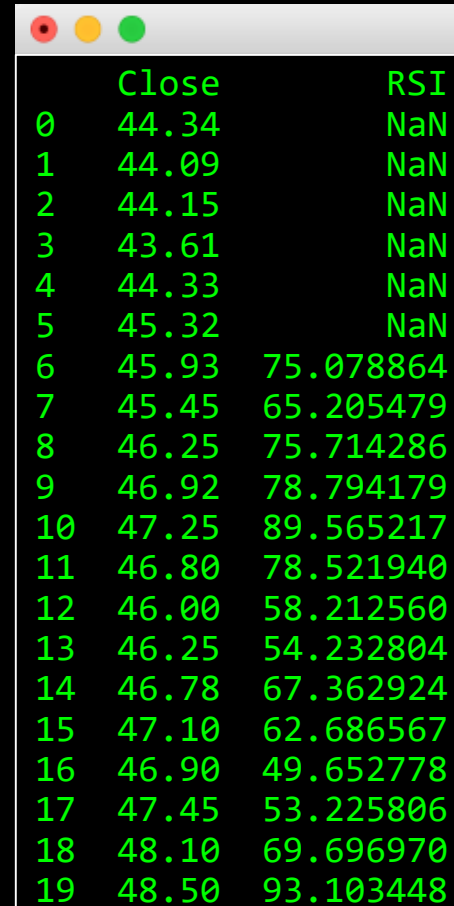
    Parameters:
    data (pd.Series): A pandas Series of closing prices.
    window (int): The number of periods to use for the RSI calculation.

    Returns:
    pd.Series: A pandas Series containing the RSI values.
    """
    delta = data.diff()
    gain = (delta.where(delta > 0, 0)).rolling(window=window).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=window).mean()

    rs = gain / loss
    rsi = 100 - (100 / (1 + rs))
    return rsi
```

```
df = pd.DataFrame({'Close': [44.34, 44.09, 44.15, 43.61, 44.33, 45.32, 45.93, 45.45,
46.25, 46.92, 47.25, 46.8, 46, 46.25, 46.78, 47.1, 46.9, 47.45, 48.1, 48.5]})
```

```
df['RSI'] = calculate_rsi(df, 7)
print(df)
```



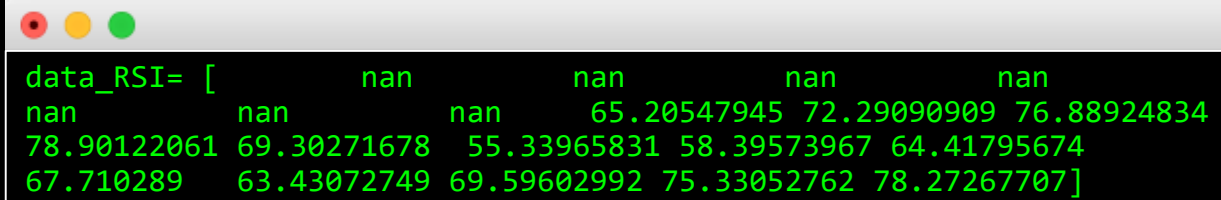
	Close	RSI
0	44.34	NaN
1	44.09	NaN
2	44.15	NaN
3	43.61	NaN
4	44.33	NaN
5	45.32	NaN
6	45.93	75.078864
7	45.45	65.205479
8	46.25	75.714286
9	46.92	78.794179
10	47.25	89.565217
11	46.80	78.521940
12	46.00	58.212560
13	46.25	54.232804
14	46.78	67.362924
15	47.10	62.686567
16	46.90	49.652778
17	47.45	53.225806
18	48.10	69.696970
19	48.50	93.103448

Example - TALIB

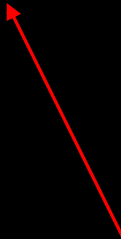
```
import numpy as np
from talib import RSI
```

```
arr = [44.34, 44.09, 44.15, 43.61, 44.33, 45.32,
45.93, 45.45, 46.25, 46.92, 47.25, 46.8, 46, 46.25,
46.78, 47.1, 46.9, 47.45, 48.1, 48.5]
```

```
data_RSI = RSI(np.array(arr), 7)
print('data_RSI=', data_RSI)
```



```
data_RSI= [      nan      nan      nan      nan      nan
nan      nan      nan      65.20547945  72.29090909  76.88924834
78.90122061  69.30271678  55.33965831  58.39573967  64.41795674
67.710289    63.43072749  69.59602992  75.33052762  78.27267707]
```



Why the result is not the same
as our previous calculation?

Example – RSI with EMA smoothing

```
data_RSI= [None, None, None, None, None, None, None,
72.290909090911, 76.88924834453827, 78.90122060961261,
69.30271677999245, 55.33965830526435,
58.395739674188725, 64.41795674477854,
67.71028899952013, 63.43072748987339, 69.59602992193464,
75.33052761587143, 78.27267706693819]
```

Now it matches with TA-LIB results

```
def my_RSI(data, n, isEMA=True):
    m = len(data)-1
    k = 1/float(n)
    RSI = [None]*m

    # error handling for insufficient data
    if m<n:
        return RSI

    # compute 1 step price difference
    delta = [data[i+1]-data[i] for i in range(0,m)]

    # create the positive gains (up) and negative gains (down) series
    gain = [abs(i) if i>0 else 0 for i in delta]
    loss = [abs(i) if i<0 else 0 for i in delta]

    avg_gain = sum(gain[0:n])/float(n)
    avg_loss = sum(loss[0:n])/float(n)

    for i in range(n,m):
        # Exponential Moving Average
        if isEMA:
            avg_gain = gain[i]*k + avg_gain*(1-k)
            avg_loss = loss[i]*k + avg_loss*(1-k)
        # Simple Moving Average
        else:
            avg_gain = (avg_gain*n - gain[i-n] + gain[i])*k
            avg_loss = (avg_loss*n - loss[i-n] + loss[i])*k
        # calculate RSI value
        if avg_loss==0:
            RSI[i] = 100.0
        else:
            RSI[i] = 100.0 - (100.0 / (1.0 + avg_gain/avg_loss))
    return RSI
```

```
import numpy as np
arr = [44.34, 44.09, 44.15, 43.61, 44.33, 45.32, 45.93, 45.45, 46.25, 46.92, 47.25, 46.8, 46, 46.25,
46.78, 47.1, 46.9, 47.45, 48.1, 48.5]
data_RSI = my_RSI(np.array(arr), 7)
print('data_RSI=',data_RSI)
```

RSI Trading Logic

- Collect daily closing price
- Calculate the latest RSI(14) value
- Order open conditions:
 - if we have **NO** outstanding position,
 - if RSI value < 30, we open a buy order
 - if RSI value > 70, we open a sell order
- Order close conditions:
 - if we have outstanding position,
 - if we previously submit a buy order and RSI value reverses back to or above 50, then close the buy order
 - if we previously submit a sell order and RSI value reverses back to or below 50, then close the sell order
- Repeat the process until the backtest period end



RSI Strategy

Backtest setup

- Instrument: SPY (<https://finance.yahoo.com/quote/SPY/>)
- Period: 2020.01.01 – 2025.12.31
- Data Interval: daily data
- Long only

Please try to implement this RSI strategy in

1. Excel
2. Backtesting.py
3. ALGOGENE

Key Takeaways

- Algo trading life cycle and backtesting process
- Candlestick charts and common patterns
- Data cleaning techniques
- Implement trading strategies on Excel and Python
- MA Cross Strategy
- Common Pitfalls in Backtesting
- Exercise – RSI Strategy